

Министерство просвещения Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Чувашский государственный педагогический университет
им. И.Я. Яковлева»

Факультет физико-математического образования,
информатики и технологий

Кафедра информатики и технологий

УТВЕРЖДАЮ
Заведующий кафедрой
_____ К. Н. Фадеева
«__» _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ (БАКАЛАВРСКАЯ) РАБОТА

Направление подготовки бакалавров
09.03.03 Прикладная информатика
Профиль: «Информационные системы и веб-приложения»

СОЗДАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ ДЛЯ ИГРЫ В ШАХМАТЫ

Научный	_____	канд. физ.-мат	_____
руководитель		наук, <u>доцент</u>	<u>А. В. Горский</u>
	подпись, дата	должность, ученая степень	инициалы, фамилия

Выпускник	_____	<u>М. С. Шмелев</u>
	подпись, дата	инициалы, фамилия

Чебоксары 2025

СОДЕРЖАНИЕ

Введение3

Глава 1. Теоретические аспекты разработки мобильных приложений6

1.1 Введение в разработку мобильных приложений**Error! Bookmark not defined.**

1.2 Анализ существующих приложений для игры в шахматы9

1.3 Описание средств разработки мобильного приложения19

1.4 Обоснование средств разработки мобильного приложения25

Глава 2. Проектирование мобильного приложения для игры в шахматы28

2.1 Описание приложения28

2.2 Объектно-ориентированное моделирование функционирования
мобильного приложения30

Глава 3. Разработка мобильного приложения28

3.1 Выбор технологий для разработки28

3.2 Описание выбранных средств для разработки мобильного приложения28

3.3 Структура базы данных28

3.4 Структура мобильного приложения28

Заключение50

Список использованной литературы52

Приложения56

Введение

Сейчас, в эпоху цифровых технологий, мобильные приложения стали неотъемлемой частью нашей жизни. Они помогают не только в быту, но и открывают новые возможности для обучения и досуга. Особенно востребованы приложения для интеллектуального отдыха, среди которых шахматы занимают особое место. Эта древняя игра, веками привлекающая миллионы людей по всему миру, обрела второе дыхание в цифровом формате.

Разработка мобильного приложения для игры в шахматы является одним из актуальных направлений в области информационных технологий и современного программного обеспечения. Шахматы – это интеллектуальная игра с многовековой историей, сохраняющая популярность во всем мире. В последние десятилетия значительный рост технологий, включая развитие мобильных платформ, позволил сделать шахматы доступными любому желающему вне зависимости от их местоположения и уровня подготовки. Процесс цифровизации шахматной игры предоставляет новые возможности для обучения, тренировки, анализа партий и даже для профессиональных состязаний.

Актуальность данной работы обусловлена несколькими важными факторами. Во-первых, растущая популярность шахмат за счет их присутствия в цифровой среде делает необходимым создание современных, функциональных и удобных мобильных приложений. Такие платформы востребованы как начинающими игроками, так и профессиональными шахматистами. Во-вторых, шахматы способствуют развитию логического мышления, стратегического планирования и навыков принятия решений, что делает их образовательным инструментом для пользователей всех возрастов. В-третьих, появление ряда шахматных приложений на рынке не отменяет потребности в разработке новых решений, которые предлагают уникальные

функции, улучшают пользовательский опыт и учитывают современные требования к безопасности данных и интерфейсу.

Теоретическая значимость работы заключается в изучении и разработке алгоритмов искусственного интеллекта, которые могут быть использованы для создания шахматных движков. Такие алгоритмы открывают возможности для анализа игровых стратегий, определения оптимальных ходов и обучения пользователей. В процессе создания шахматного приложения исследуются подсистемы взаимодействия с пользователем, механизмы отображения шахматных позиций и пути интеграции шахматного движка с визуальным интерфейсом. Это дает теоретическую основу для разработки подобных приложений, включая решения для других видов интеллектуальных игр.

Практическая значимость работы состоит в создании мобильного приложения для игры в шахматы, которое сможет удовлетворить как потребности любителей шахмат, так и профессионалов. Разработанное приложение станет инструментом для проведения игр, изучения новых стратегий и анализа ошибок. Возможность играть в любое удобное время, наличие функций обучения и встроенного анализа партий делает такое приложение полезным не только для развлечения, но и для повышения уровня навыков пользователя. Кроме того, результаты данной работы могут быть применены в создании других обучающих приложений, где важна симуляция сложных интеллектуальных процессов.

Цель данной работы заключается в разработке высококачественного, функционального мобильного приложения для игры в шахматы, которое будет отвечать современным техническим требованиям, учитывать запросы пользователей и предоставлять разнообразные возможности для игры и обучения.

Объект исследования – это процесс создания мобильных приложений для интеллектуальных игр, в частности для игры в шахматы.

Предмет исследования – методы проектирования и разработки шахматного мобильного приложения, включая программное обеспечение, пользовательский интерфейс, алгоритмы работы шахматного движка и подходы к обеспечению удобного взаимодействия с пользователем.

Таким образом, проект направлен на объединение теоретических знаний в области программирования и компьютерных шахмат с практическими потребностями пользователя, чтобы создать современный и востребованный инструмент для игры в шахматы.

Глава 1 Теоретические аспекты разработки мобильных приложений

1.1 Введение в разработку мобильных приложений

Мобильные приложения – это программы, созданные для работы на мобильных гаджетах, например, на смартфонах и планшетах. Установленные на этих устройствах, они предоставляют пользователям широкий спектр возможностей, доступных в любое время и независимо от местоположения.

Мобильные приложения можно разделить на несколько типов:

- нативные приложения;
- кроссплатформенные приложения;
- веб-приложения.

Нативные приложения (Native Apps). Нативные приложения разрабатываются с использованием языков программирования и инструментов, специфичных для конкретной операционной системы. Для iOS это обычно Swift или Objective-C, для Android – Kotlin или Java. Эти приложения устанавливаются непосредственно на устройства и могут использовать все возможности операционной системы. К преимуществам нативных приложений можно отнести их высокую производительность ввиду их оптимизации для конкретного устройства, возможность использовать различные функции устройства (камера, GPS, акселерометр, микрофон), а также интуитивно понятный пользовательский интерфейс. Из недостатков таких приложений можно выделить то, что они требуют отдельной разработки и поддержки для каждой платформы.

Кроссплатформенные приложения (Cross-Platform Apps). Кроссплатформенные приложения разрабатываются с использованием фреймворков, которые позволяют писать один и тот же код для нескольких операционных систем. Наиболее популярные фреймворки включают React Native и Flutter. Эти приложения могут быть развернуты как на iOS, так и на Android с минимальными изменениями кода. Ниже представлены преимущества кроссплатформенных приложений:

- разработка ведется с использованием одного кода, который затем компилируется для различных операционных систем;
- низкие затраты на разработку и поддержку.

Недостатки кроссплатформенных приложений:

- ограничения в производительности;
- ограниченный доступ к функциям устройства.

Веб-приложения (Web Apps). Веб-приложения работают через интернет-браузер и не требуют установки на устройство. Они разрабатываются с использованием веб-технологий, таких как HTML, CSS и JavaScript. Веб-приложения могут быть доступны на любом устройстве с интернет-браузером, что делает их платформонезависимыми. Преимуществами веб-приложений являются их независимость от платформы и скачивание обновлений на стороне сервера. К их недостаткам можно отнести ограниченные возможности в доступе к устройству и медленную работу по сравнению с нативными приложениями.

Далее будут приведены платформы для мобильной разработки. Мобильный рынок преимущественно разделен между двумя основными операционными системами(см. рисунок 1.1):

1. Android – разработанная Google операционная система с открытым исходным кодом, используемая множеством производителей устройств: Samsung, Huawei, Xiaomi и др. Android отличается разнообразием устройств, доступной ценой и широкой пользовательской базой.

2. iOS – проприетарная операционная система, разработанная Apple и используемая исключительно на устройствах компании (iPhone, iPad). iOS известна своим вниманием к безопасности, стабильностью и высокими стандартами качества приложений.



Рисунок 1.1 – Операционные системы мобильных приложений

Выбор платформы для разработки. При выборе платформы для разработки приложения необходимо учитывать несколько факторов:

- целевая аудитория: если целевая аудитория преимущественно использует Android или iOS, имеет смысл сосредоточиться на соответствующей платформе.
- ресурсы и бюджет: разработка под каждую платформу требует определенных ресурсов. Нативная разработка для обеих платформ одновременно может быть дорогостоящей.
- функциональные требования: некоторые функции могут быть проще реализованы на одной платформе по сравнению с другой из-за особенностей системы или доступных API.

Языки программирования и инструменты (см. рисунок 1.2). Для каждой платформы используются определенные языки программирования и инструменты разработки.

- Android:
 - Java: традиционный язык для разработки Android-приложений. Обладает широкой базой знаний и сообществом разработчиков.
 - Kotlin: более современный язык, полностью совместимый с Java, отличающийся лаконичным синтаксисом и повышенной безопасностью кода.
 - Android Studio: официальная интегрированная среда разработки (IDE), предоставляющая инструменты для кодирования, отладки и тестирования приложений.

- iOS:

- Objective-C: исторический язык для разработки на платформе iOS. Менее распространен в новых проектах.

- Swift: современный язык программирования, разработанный Apple, сочетающий простоту и мощь. Стал основным языком для разработки новых приложений.

- Xcode: официальная IDE для разработки под iOS, предоставляющая полный набор инструментов, включая симулятор устройств, инструменты для UI-дизайна и отладки.

Сравнение технологий мобильной разработки

				
Языки	Swift, Objective-C	Kotlin, Java	JavaScript, TypeScript	Dart
Платформы	iOS, MacOS, tvOS, WatchOS, iPadOS	Android, AndroidTV	iOS, Android, tvOS, AndroidTV	iOS, Android
Компании-разработчики	Apple	JetBrains	Facebook	Google

Рисунок 1.2 – Сравнение технологий мобильной разработки

1.2 Анализ существующих приложений для игры в шахматы

Составим список наиболее популярных на сегодняшний день приложений для игры в шахматы и сделаем краткий обзор каждого, чтобы определить их преимущества и недостатки.

Chess.com – крупнейшая платформа для игры в шахматы, где пользователи со всего мира могут соревноваться онлайн, а также изучать шахматы через обучающие видео и статьи. Логотип и дизайн сайта изображены на рисунках 1.3 и 1.4.



Рисунок 1.3 – Логотип приложения chess.com



Рисунок 1.4 – Внешний вид веб-сайта приложения

Chess.com часто называют социальной сетью для любителей шахмат, где разработчики уделили значительное внимание созданию удобного и привлекательного интерфейса для пользователя. При этом функционал платформы также впечатляет. Основные возможности приложения:

- шахматные партии: сайт предлагает различные режимы игры, включая стандартные, быстрые шахматы, королевские шахматы и другие. Игроки имеют возможность сражаться друг с другом онлайн, выбирая уровень сложности против бота или принимая участие в играх других пользователей;
- анализ партий: chess.com предоставляет инструменты для разбора сыгранных игр, что помогает пользователям улучшать свои умения и выявлять ошибки в своей игре. Также на платформе можно смотреть трансляции профессиональных игр и обсуждать их с другими пользователями;
- обучение: приложение предоставляет широкий спектр учебных материалов, таких как видеоуроки, статьи и задачи, которые охватывают начальные и продвинутые уровни, что позволяет пользователям совершенствовать свои навыки в зависимости от уровня их игры.

Далее будут перечислены достоинства и недостатки данного приложения.

Достоинства chess.com:

- интуитивный интерфейс: Платформа снабжена удобным и эстетически приятным интерфейсом, что облегчает ее использование как для новичков, так и для опытных игроков;
- обширные обучающие ресурсы: Приложение предоставляет множество учебных материалов, от видеоуроков до статей и шахматных задач, которые охватывают все уровни подготовки;
- аналитические инструменты: Возможность анализа игр помогает выявлять ошибки и совершенствовать стиль игры;
- разнообразие режимов игры: От стандартных шахмат до уникальных форматов и уровней сложности ботов, что позволяет удовлетворить интересы всех игроков.

Недостатки chess.com:

- платная подписка: Некоторые функции и возможности доступны только пользователям платной подписки;
- перегруженность интерфейса: Из-за большого количества функций иногда может показаться, что интерфейс перегружен информацией;
- сетевая зависимость: Для полноценного использования всех возможностей платформы необходимо стабильное интернет-соединение.

Описав сильные и слабые стороны можно сказать, что chess.com действительно является хорошей платформой для игры, которая может удовлетворить спрос многих пользователей, но перечисленные слабые стороны существенно портят имидж приложения.

Lichess.org. Данное приложение предоставляет возможность бесплатно играть в шахматы онлайн с другими пользователями в режиме реального времени, предлагать обучающие материалы для изучения шахмат и участвовать в турнирах. Lichess.org считается одной из самых популярных платформ для онлайн-игры в шахматы. Логотип и дизайн сайта можно увидеть на рисунках 1.5 и 1.6.

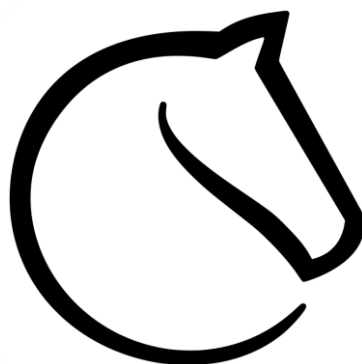


Рисунок 1.5 – Логотип приложения Lichess.org

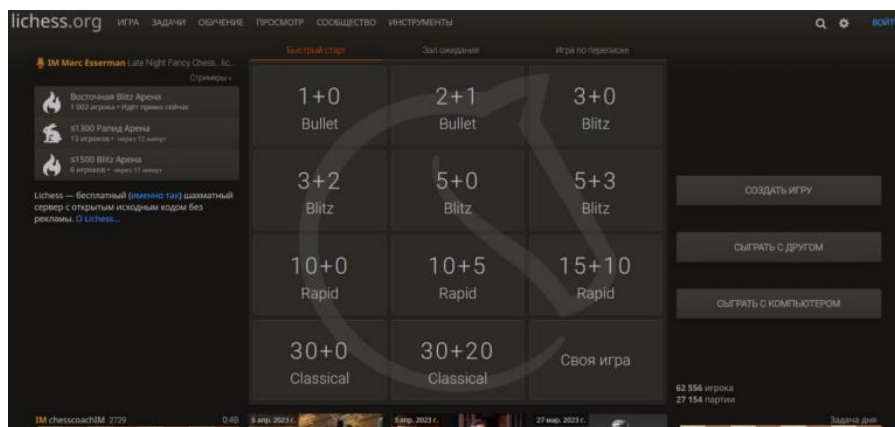


Рисунок 1.6 – Внешний вид сайта приложения Lichess.org

Перечислим основной функционал Lichess.org:

- игра против других пользователей: В приложении реализована возможность игры в шахматы в реальном времени с соперниками со всего мира. Доступны разные режимы игры, включая классику, блиц и пулю;
- обучение шахматам: Платформа предлагает многочисленные обучающие материалы, включая задачи для решения, видеоуроки и анализ партий;
- персональные настройки: Индивидуализация игрового опыта с помощью настройки внешнего вида доски, фигур и других параметров;
- форумы и сообщества: Возможность общения с другими любителями шахмат в специализированных форумах и клубах.

Описав функциональные возможности приложения выделим его достоинства и недостатки.

Достоинства Lichess.org:

- бесплатность: Все функции на платформе доступны бесплатно, включая анализ партий и обучение;
- отсутствие рекламы: На сайте нет рекламы, что обеспечивает более комфортный игровой процесс;
- открытый исходный код: Lichess.org является проектом с открытым исходным кодом, что позволяет сообществу улучшать и развивать платформу;

- широкие возможности анализа: Доступен мощный анализ партий с использованием современных шахматных движков;
- большое количество режимов игры: Возможность выбора из разных режимов и контролей времени.

Недостатки Lichess.org:

- интернет-зависимость: Для использования некоторых функций требуется постоянное подключение к интернету;
- меньшая узнаваемость: По сравнению с коммерческими шахматными платформами, такими как Chess.com, Lichess может быть менее известным некоторым пользователям;
- ограниченная поддержка: Как некоммерческий проект, Lichess может предлагать ограниченную поддержку пользователей по сравнению с платными платформами.

Lichess предлагает все основные функции, необходимые для игры в шахматы онлайн. Однако отмеченные проблемы приводят к значительному оттоку пользователей и мешают считать этот продукт совершенным решением для шахматистов.

Chess24.com. Популярная онлайн-платформа для игры в шахматы и обучения. На сайте предоставляется широкий спектр возможностей для шахматистов всех уровней. Логотип приложения и его внешний вид представлены на рисунках 1.7 и 1.8.



Рисунок 1.7 – Логотип приложения Chess24

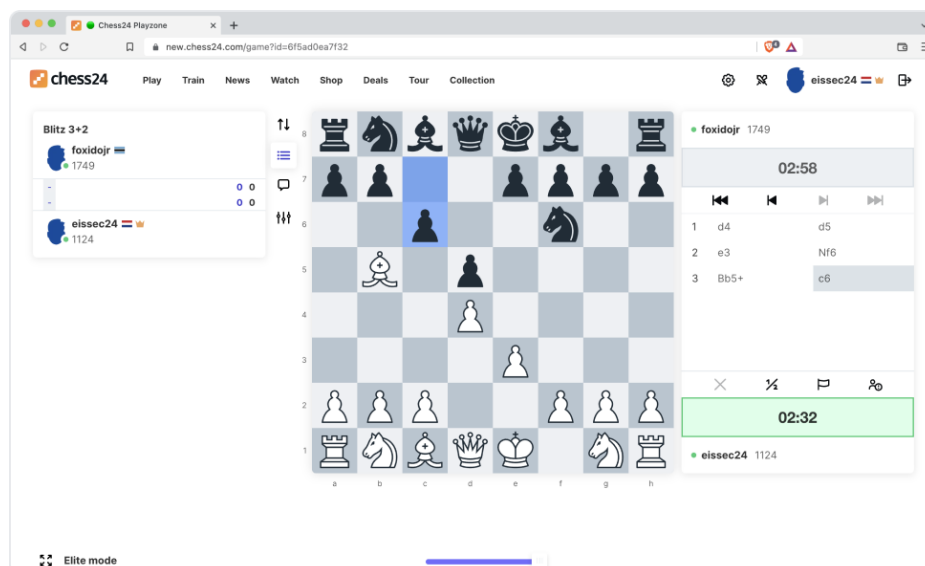


Рисунок 1.8 – Внешний вид приложения Chess24

Ниже в виде списка представлены основные функциональные возможности приложения Chess24:

- онлайн-игры – сайт предлагает возможность играть в шахматы через интернет с соперниками из различных стран, а также имеется опция игры с искусственным интеллектом;
- статистика и аналитика – сайт предлагает исчерпывающую статистику и аналитическую информацию по партиям, турнирам и участникам, что помогает шахматистам совершенствовать свои умения и тактику;
- магазин шахматных товаров – в веб-приложении можно купить шахматы, комплекты, книги, DVD и другие связанные с игрой товары;
- мобильное приложение – chess24.com предлагает мобильное приложение для iOS и Android, что обеспечивает доступ к сервисам платформы в любом месте;
- языковая поддержка – сайт снабжен многими языковыми версиями, включая английский, испанский, немецкий, французский, русский и другие.

Описав функциональные возможности данного приложения перейдем к выявлению его достоинств и недостатков.

Достоинства chess24:

- многофункциональность: Chess24 предлагает широкий ассортимент возможностей, таких как онлайн-игры, обучающие материалы, трансляции турниров и аналитика, всё в одном месте;
- обучение: Доступны разнообразные материалы для обучения, включая видеоуроки и занятия с тренерами;
- мобильное приложение: Имеется приложение для iOS и Android, чтобы пользоваться сервисом в любое время и в любом месте;
- международное сообщество: Веб-сайт поддерживается на нескольких языках и позволяет общаться и играть с пользователями со всего мира;
- трансляции турниров: Возможность смотреть живые трансляции турниров и получать комментарии от профессионалов.

Недостатки chess24:

- платный контент: Для доступа к определённым функциям и материалам обучение может потребовать подписку или разовую плату;
- технические сбои: Иногда пользователи могут сталкиваться с проблемами загрузки или работы сайта;
- конкуренция: На рынке имеется множество других шахматных платформ, которые могут предлагать аналогичные или более продвинутые функции;
- перегруженность: Из-за обилия функций новички могут найти интерфейс несколько сложным для навигации и использования.

В общем, chess24.com представляет собой хорошую онлайн-платформу для шахматистов, но имеет некоторые ограничения, которые могут снизить удобство её использования для части пользователей. Платформу можно больше рассматривать как источник информации. Если вы хотите узнать о текущих событиях в шахматном мире или посмотреть несколько обучающих уроков, chess24.com предоставит такие возможности. Однако для сетевой

игры в шахматы у приложения довольно ограниченные функции, что делает его менее подходящим для широкого использования.

Chesscube.com – это платформа, где можно сразиться в шахматы с другими пользователями в онлайн-режиме, а также смотреть трансляции шахматных турниров и обучающие видео. Внешний вид и логотип приложения представлены на рисунках 1.9 и 1.10.



Рисунок 1.9 – Логотип приложения chesscube



Рисунок 1.10 – Внешний вид приложения chesscube

Перейдем к перечислению основных особенностей приложения:

- онлайн-игра: Возможность играть в шахматы с другими пользователями в режиме реального времени;
- трансляции турниров: Просмотр онлайн-трансляций различных шахматных турниров;

- лояльная пользовательская база: Активное сообщество игроков, с которыми можно соревноваться и общаться;
- интуитивный интерфейс: Удобный и понятный интерфейс, который облегчает навигацию по сайту.

Далее будут представлены достоинства и недостатки данного приложения.

Достоинства chesscube:

- простота в использовании: Интуитивно понятный интерфейс, даже для новичков;
- быстрая игра: Возможность находить оппонентов за считанные секунды;
- награды и виртуальная валюта (Cubits): Возможность получать виртуальную валюту для участия в турнирах и покупок;
- разные форматы шахмат: Включены как классические партии, так и более быстрые форматы, например, блиц;
- социальный элемент: Чат и социальные функции для общения с игроками.

Недостатки chesscube:

- ограничения для бесплатных пользователей: Некоторые функции доступны только за реальную или виртуальную валюту (Cubits);
- снижение активности: Со временем популярность платформы снизилась, из-за чего стало меньше активных игроков;
- ограниченная функциональность по сравнению с конкурентами: Например, меньше возможностей для анализа партий, чем на сайтах вроде Chess.com или Lichess.org;
- технические проблемы: Пользователи иногда жалуются на нестабильную работу приложения.

Анализируя предлагаемые функции и уникальные характеристики этого сайта, можно отметить, что в начале эпохи веб-программирования он по праву занимал ведущие позиции. Очевидно, что современные лидеры в этой области переняли некоторые функции этого предшественника, что свидетельствует о продолжающейся актуальности отдельных элементов сайта. Это также указывает на то, что такие продукты все еще служат вдохновением для создания собственных аналогов в наше время.

1.3 Описание средств разработки мобильного приложения

Разработка мобильных приложений включает использование различных технологий, инструментов и библиотек, которые составляют так называемый технологический стек. Успешная реализация проекта во многом зависит от правильно подобранного стека технологий, который будет соответствовать требованиям проекта, целевой платформе и особенностям приложения. В данной подглаве будут подробно рассмотрены ключевые компоненты технологического стека, используемых при создании мобильных приложений.

Технологический стек для любого мобильного приложения можно условно разделить на следующие слои:

1. Фронтенд (Frontend) – пользовательская часть приложения, с которой взаимодействует конечный пользователь.
2. Бэкенд (Backend) – серверная сторона приложения, которая отвечает за обработку, хранение данных и выполнение бизнес-логики.
3. Инструменты разработки – среды разработки (IDE), системы контроля версий, отладчики и эмуляторы.
4. Инфраструктура и базы данных – серверы, хранилища данных, облачные технологии и API.
5. Инструменты тестирования – программное обеспечение для автоматизации тестирования и устранения ошибок.

Ниже подробно рассмотрен каждый слой технологического стека для разработки мобильных приложений.

Фронтенд: разработка пользовательского интерфейса. Пользовательский интерфейс – это та часть приложения, с которой взаимодействует конечный пользователь. Его разработка во многом зависит от выбранного подхода: будете ли вы создавать нативное приложение для конкретной платформы, к примеру, Android или iOS, или же выберете кроссплатформенный подход, чтобы разработать единую программу, работающую на обеих системах.

Для создания приложений на Android традиционно применяются языки программирования Java и Kotlin. Сегодня Kotlin стал основным выбором среди Android-разработчиков благодаря своей современности, удобству и повышенной безопасности. Для разработки приложений используется Android Studio, предоставляющая редактор кода, средства отладки, тестирования и эмулятор устройств.

Что касается операционной системы iOS, здесь основными языками остаются Objective-C и Swift. Разработчики предпочитают Swift из-за его скорости и простоты использования. Для iOS предназначена официальная среда разработки Apple – Xcode, которая предлагает все необходимые инструменты для работы с интерфейсом, тестирования и подготовки приложения к выпуску в App Store.

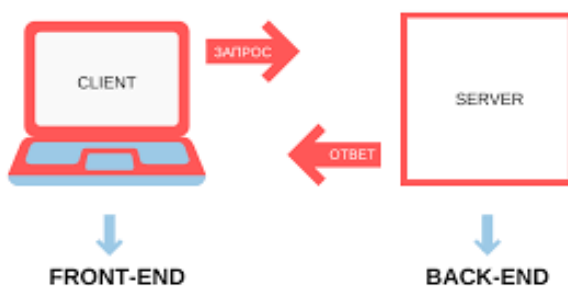


Рисунок 1.11 – Взаимодействие клиентской и серверной части мобильного приложения

Серверная часть приложения, или бэкенд, играет ключевую роль в функционировании мобильного приложения, особенно когда требуется хранение данных, сложная бизнес-логика или обмен информацией между пользователями. Бэкенд обрабатывает запросы, обеспечивает доступ к базам данных и интеграцию с внешними сервисами. Для разработки серверной части используют различные языки программирования. Так, Python в сочетании с фреймворками Django или Flask прекрасно подходит для создания безопасных и функциональных серверов. JavaScript с платформой Node.js помогает разрабатывать производительные и масштабируемые системы, а Go становится предпочтительным выбором для высоконагруженных приложений. Java вместе с фреймворком Spring или PHP с Laravel также остаются надежными вариантами для разработки.

Для хранения данных используются базы данных (см. рисунок 1.12). Когда структура данных четко организована в виде таблиц и остается стабильной, оптимальным выбором будет реляционная база данных, такая как MySQL, PostgreSQL или SQLite. Если же данные часто изменяются или требуется высокая скорость обработки больших объемов информации, предпочтение отдают NoSQL решениям, например MongoDB или Firebase Realtime Database. Кроме того, существуют облачные базы данных, такие как Firestore или Azure Cosmos DB, которые избавляют разработчиков от необходимости управлять настройкой и обслуживанием серверов.



Рисунок 1.12 – Взаимодействие между клиентской, серверной частью и базой данных мобильного приложения

Средства разработки. Выбор подходящих инструментов необходим для создания мобильного приложения. Первым и наиболее важным компонентом являются среды разработки. Для этого используйте Android Studio, Xcode или универсальные решения, такие как Visual Studio Code и IntelliJ IDEA, которые предоставляют разработчикам удобные инструменты для написания, отладки и тестирования кода.

Для командной работы используются системы контроля версий, такие как Git (см. рисунок 1.13), где работа часто организуется с помощью платформ вроде GitHub, GitLab или Bitbucket. Эти системы позволяют отслеживать изменения и управлять разными версиями проекта. Помимо этого, в разработке активно применяются системы для автоматизации процессов тестирования и развёртывания (CI/CD), например, Jenkins или GitHub Actions.

Не стоит забывать про дизайн-приложения, который разрабатывается в профессиональных инструментах для проектирования интерфейсов, таких как Figma, Sketch или Adobe XD.



Рисунок 1.13 – Логотип системы контроля версий git

Инструменты тестирования. Перед релизом приложения, оно обязательно должно пройти тестирование для устранения багов и проверки производительности. Самый первый этап – это тестирование приложения на разных устройствах и платформенных версиях. Например, Android имеет огромное множество смартфонов с разными характеристиками, что требует тщательной проверки. iOS в этом плане проще – экосистема Apple имеет значительно меньшее количество устройств.

Для автоматизации этого процесса используются такие инструменты, как Appium(см. рисунок 1.14) и Selenium, а для более глубокого тестирования производительности подойдут Firebase Test Lab или специализированные фреймворки, встроенные в Android Studio и Xcode. На этапе разработки полезны встроенные симуляторы устройств, которые позволяют протестировать приложение без необходимости подключения реального смартфона.



Рисунок 1.14 – Инструмент автоматизированного тестирования appium

Облачные технологии и инфраструктура. После завершения разработки мобильного приложения важно развернуть его серверную часть и подготовить к возможному увеличению нагрузки. Для этих целей часто применяются облачные сервисы, такие как Amazon Web Services (см. рисунок 1.15) (AWS), Google Cloud или Microsoft Azure. Эти платформы предоставляют удобные инструменты для управления серверами, хранения данных и доставки контента.



Рисунок 1.15 – Облачные сервисы Amazon

Настройка инфраструктуры может включать использование технологий контейнеризации, например, с Docker(см. рисунок 1.16), который позволяет поддерживать приложение в изолированной среде. Для эффективного

управления контейнерами часто используется Kubernetes(см. рисунок 1.17), который облегчает процессы масштабирования и распределения нагрузки.

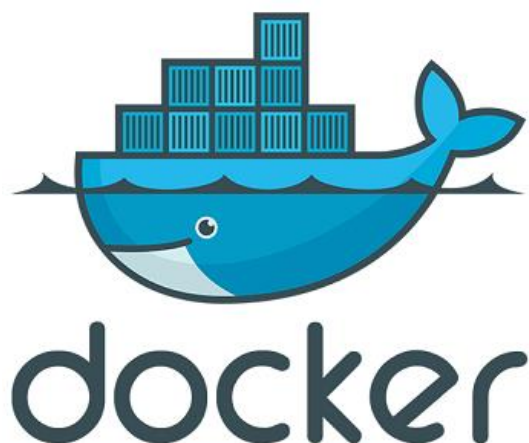


Рисунок 1.16 – ПО Docker для автоматизации развертывания и управления приложениями



Рисунок 1.17 – Оркестратор контейнеризованных приложений kubernetes

1.4 Обоснование средств разработки мобильного приложения

Мобильное приложение для игры в шахматы будет написано на языке python с использованием фреймворка Kivy(см. рисунок 1.18).



Рисунок 1.18 – Фреймворк kivy

Обоснованием выбора Kivy в качестве основного средства разработки мобильного шахматного приложения на Python является его кроссплатформенность, гибкость и широкий функционал для создания пользовательских интерфейсов. Kivy предоставляет разработчикам богатый набор инструментов, включая виджеты и возможности для обработки ввода с экрана, что значительно упрощает процесс создания интерфейса для шахматной игры. Это позволяет избежать выполнения большого количества рутинных задач при проектировании и сосредоточиться на логике и дизайне приложения. Благодаря этому сокращается общее время разработки без ущерба для качества конечного продукта.

Использование Kivy позволяет разработать приложение, которое работает на таких платформах, как Android и iOS, без необходимости переписывать код для каждой операционной системы. Это особенно ценно для мобильных приложений с широкой аудиторией. Кроме того, библиотека поддерживает такие функции, как создание анимаций и работа с графикой, что может быть полезно для реализации шахматной доски, фигур и других визуальных элементов приложения.

Благодаря поддержке Python, Kivy ориентирован как на начинающих разработчиков, так и на профессионалов, предоставляя интуитивно понятный

механизм создания дизайна через язык разметки KV, а также возможности программирования функциональности непосредственно в коде Python. Действительно, для начинающих разработчиков Kivy может быть особенно полезен, поскольку предлагает богатую систему документации и активное сообщество, готовое помочь в решении возникающих вопросов.

Для упрощения дальнейшего развертывания и распространения приложения в экосистеме мобильных устройств используются дополнительные инструменты, такие как PyInstaller и Buildozer. С помощью PyInstaller возможно формирование исполняемых файлов для настольных систем, а Buildozer обеспечивает упаковку приложения для Android или iOS. Это дает разработчику возможность сосредоточиться на логике приложения без необходимости изучения сложных инструментов создания установочных пакетов или настройки окружения.

Решение использовать язык Python позволяет исключить необходимость изучения более сложных языков, таких как Java или Swift, и преимущественно сосредоточиться на основах разработки логики шахматной игры. К тому же в Python имеются готовые библиотеки, такие как chess-python, которые можно использовать для реализации правил игры, проверки состояния доски и обработки возможных ходов. Это исключает необходимость написания сложного кода для реализации базовых функций шахматной логики и ускоряет процесс разработки. Популярность Kivy и Python обусловлены их открытым исходным кодом, что дает возможность постоянно развивать проект, используя все более новые и актуальные инструменты. Комплексный подход, предлагаемый этими инструментами, обеспечивает высокую производительность, интуитивно понятный интерфейс и надежность создаваемого мобильного приложения.

Для мобильного шахматного приложения на Python с использованием Kivy в качестве базы данных будет выбрана PostgreSQL, так как это встраиваемая база данных, которая не требует отдельного сервера, что делает её отличным выбором для мобильных приложений, где важно

минимизировать использование ресурсов и обеспечить небольшую загрузку приложения. Также PostgreSQL легко интегрируется с Python, и для работы с ней доступны библиотеки, которые упрощают задачи, связанные с управлением базой данных, такие как CRUD-операции (создание, чтение, обновление и удаление данных). В целом использование данной базы данных это надежный выбор для приложений, где потеря данных может оказать серьёзное влияние на пользовательский опыт. База данных поддерживает атомарные транзакции, что является важным фактором для безопасности и целостности данных.

Глава 2 Проектирование мобильного приложения для игры в шахматы

2.1 Описание приложения

Мобильное приложение для игры в шахматы представляет собой многофункциональную платформу, которая объединяет возможности как для любителей шахмат, так и для профессиональных игроков. Приложение будет разработано с учетом простоты использования, интуитивно понятного интерфейса и поддержки различных режимов игры. Основной задачей приложения станет предоставление пользователям удобного доступа к шахматной игре как в одиночном режиме, так и в формате игры с другими пользователями через интернет.

Приложение будет состоять из нескольких ключевых страниц и модулей:

1. Главная страница

- На главной странице пользователь сможет выбрать режим игры: одиночный (игра против искусственного интеллекта), сетевой (онлайн-игра с другим пользователем), а также режим изучения (анализ партий и шахматных комбинаций).

- Здесь же будет представлена краткая сводка о текущем рейтинге игрока, истории последних партий и персональных достижениях.

2. Страница "Игра"

- Этот экран станет центральным местом реализации игры. Интерфейс страницы включает шахматную доску, которую можно повернуть в зависимости от стороны игрока (белые или черные), счетчик времени для контроля партий с ограниченным временем и информационные панели с ходами.

- Пользователю будет предоставлена возможность выбора уровня сложности в одиночной игре, а также опции отмены и повторения последнего хода.

3. Сетевая игра

- В этом разделе будет реализована система подключения к серверу для игры с другими пользователями. Игроки смогут создавать комнаты для частных партий или подключаться к открытым матчам через систему автоматического поиска соперников на основе их рейтинга.

- Также добавляется текстовый чат, чтобы игроки могли обмениваться сообщениями во время матча.

4. Личный кабинет

- Здесь пользователь сможет просматривать свою статистику: количество выигранных и проигранных партий, текущий рейтинг, список достижений и наград.

- Доступны настройки профиля, включая изменение аватара, имени пользователя и предпочтений (темная/светлая тема, уведомления).

5. Раздел обучения

- Раздел предложит обучение правилам шахмат, базовые стратегии, а также сложные шахматные задачи. Пользователи смогут просматривать известные партии через анализ ходов.

- Также можно выделить функционал "анализ игры", который позволит пользователю загрузить ранее сыгранную партию и получить детальный разбор с комментариями ИИ.

6. История матчей

- Страница с историей сыгранных партий, включая возможность просмотра каждого хода и анализа ошибок.

- Пользователь сможет фильтровать игры по дате, типу матча (онлайн, против ИИ, тренировка) или результату.

7. Настройки приложения

- В разделе настроек пользователь сможет изменить предпочтения в отображении доски, выбрать цвет фигур, настроить время партий и другие параметры интерфейса.

Серверная часть приложения будет отвечать за обработку всех запросов, связанных с хранением данных, доступом к онлайн-матчам и обеспечением рейтинговой системы. Сервер будет реализован с использованием современных веб-технологий для обеспечения высокой производительности и масштабируемости. Разработка серверной части будет вестись с применением современных технологий, таких как Node.js или Python (Django), что обеспечит быстрый отклик системы. Для базы данных можно использовать PostgreSQL или MongoDB, а для реального времени – WebSocket или SocketIO.

2.2 Объектно-ориентированное моделирование функционирования мобильного приложения

Диаграммы играют ключевую роль в моделировании функционирования мобильных приложений. Они помогают визуализировать структуру и поведение приложения, упрощают процесс разработки, способствуют лучшему пониманию требований и взаимодействий между различными компонентами. Диаграммы позволяют выявить и устранить потенциальные проблемы на ранних стадиях проектирования, обеспечивая более эффективную и надежную реализацию приложения.

На рисунке 2.1 представлена диаграмма вариантов использования, отображающая взаимодействие акторов с приложением. Неавторизованный пользователь имеет возможность авторизации в приложении или создать новую учетную запись. Авторизованный пользователь может перейти в личный кабинет, начать игру, просматривать игры других пользователей в режиме реального времени, а также перейти в чат для общения. Администратор приложения после авторизации обладает широкими правами: он может управлять пользователями, удалять их или добавлять новых, а также очистить чат от сообщений, не соответствующих правилам сообщества.

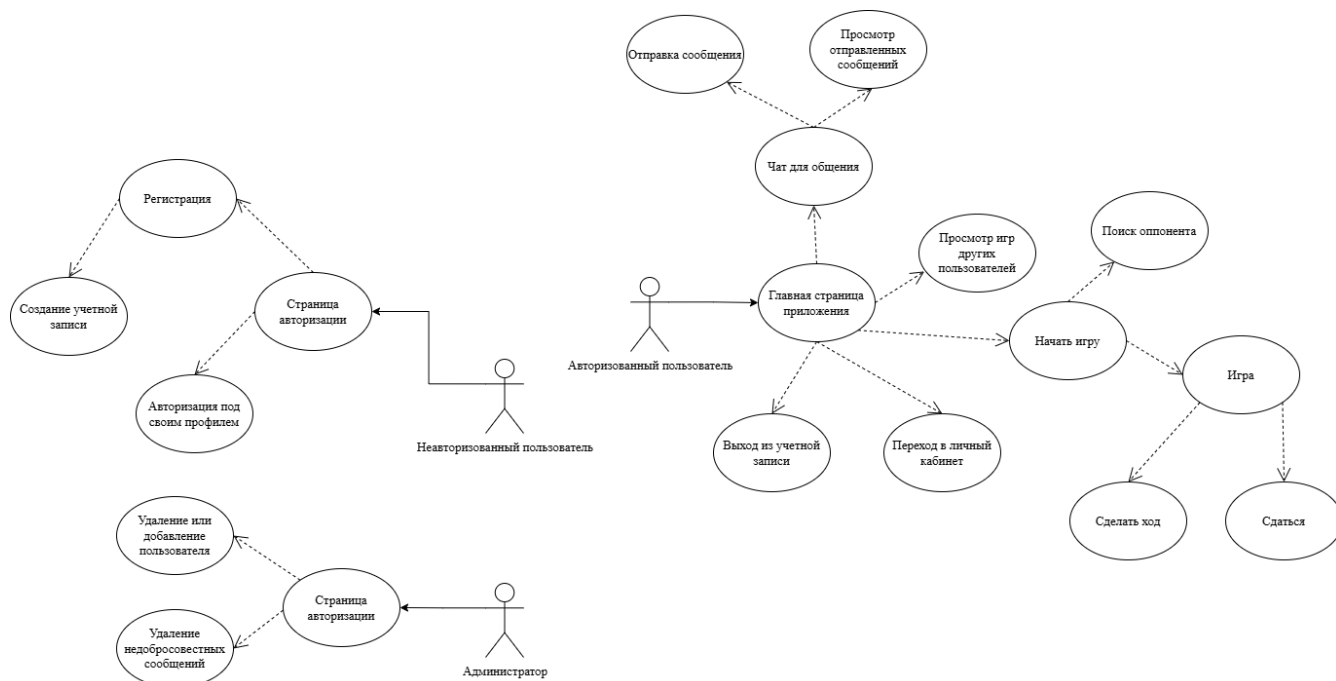


Рисунок 2.1 – Диаграмма вариантов использования

На рисунках 2.2 и 2.3 представлены диаграммы состояний, отображающие различные состояния системы в процессе авторизации в приложении и при поиске игры пользователем.

Пользователь приложения проходит процесс авторизации. При успешном выполнении этих действий, пользователя перенаправляет на главную страницу приложения.

Авторизованный пользователь заходит на главную страницу и активирует поиск игры. Затем система на веб-сервере начинает поиск подходящего соперника. Если соперник не найден, то поиск продолжается. Когда соперник найден, создается игровая сессия, и пользователь получает уведомление о готовности игры. Пользователь может перейти на страницу игры, чтобы начать играть. Если пользователь решает продолжать ждать, поиск игры повторяется.

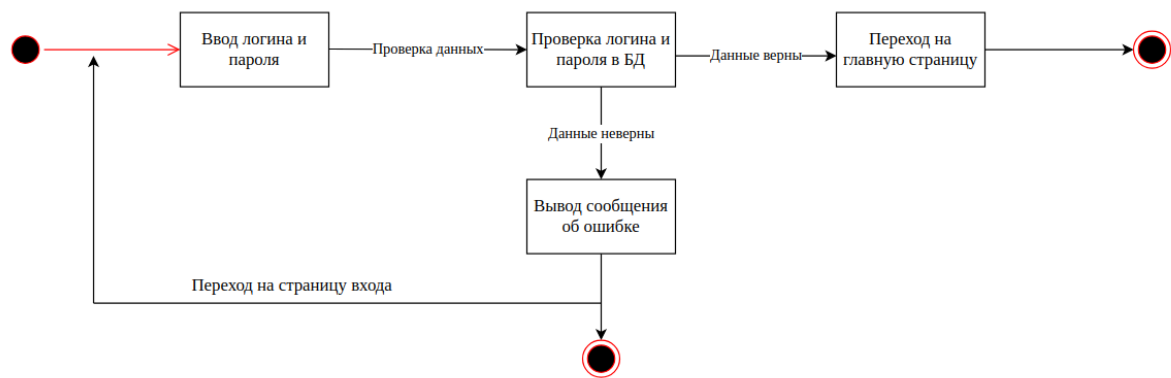


Рисунок 2.2 – Диаграмма состояний «Авторизация»

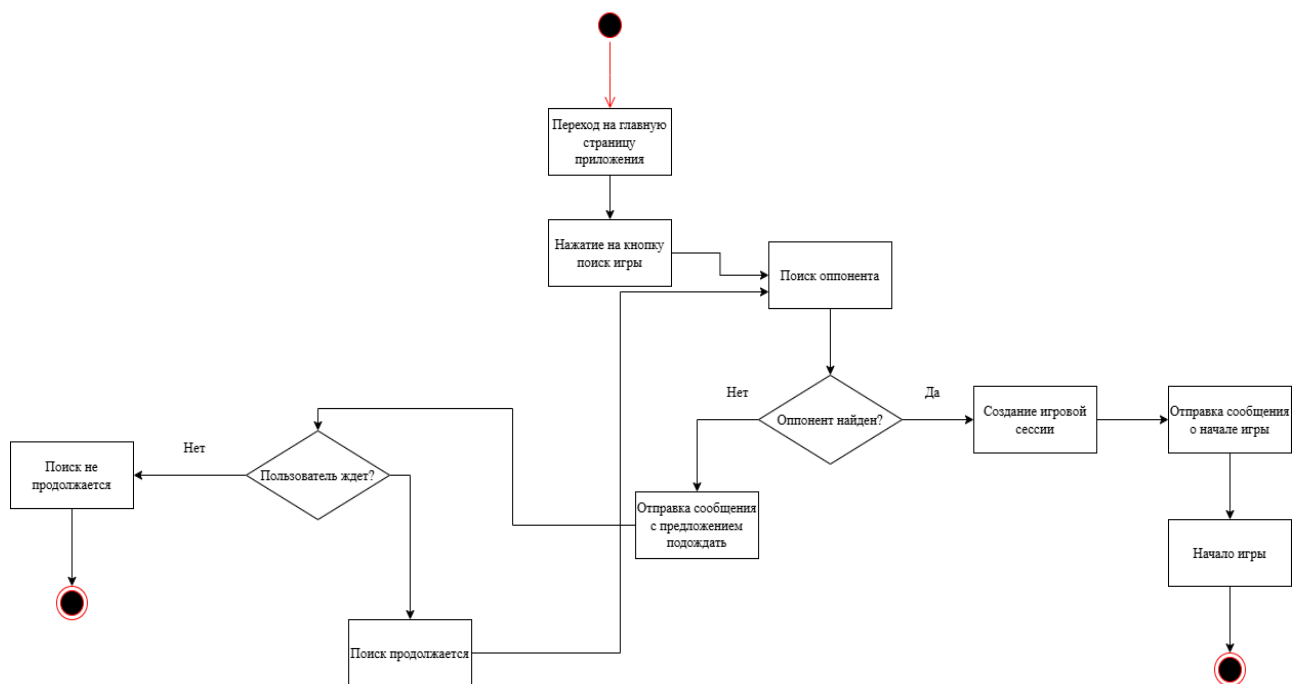


Рисунок 2.3 – Диаграмма состояний «Поиск игры»

На рисунке 2.4 представлена диаграмма классов, показывающая структуру классов приложения и их взаимосвязи. Диаграмма демонстрирует сложные отношения и зависимости между различными элементами приложения, такими как пользователи, партии, достижения и сообщения.



Рисунок 2.4 – Диаграмма классов

В данной диаграмме показано, что пользователь может иметь несколько достижений, а каждое достижение связано только с одним пользователем. Пользователь может отправлять несколько сообщений, при этом каждое сообщение принадлежит одному пользователю. В каждой партии участвуют два пользователя, причем для каждой партии указывается первый и второй пользователь, а также кто из них победил, но одна партия обязательно связана с двумя пользователями, а каждый пользователь может принять участие во множестве партий.

На рисунке 5 представлена диаграмма пакетов, демонстрирующая организацию модулей приложения и их взаимодействие. Диаграмма помогает понять, как различные части приложения связаны и взаимодействуют для выполнения функций и задач системы.

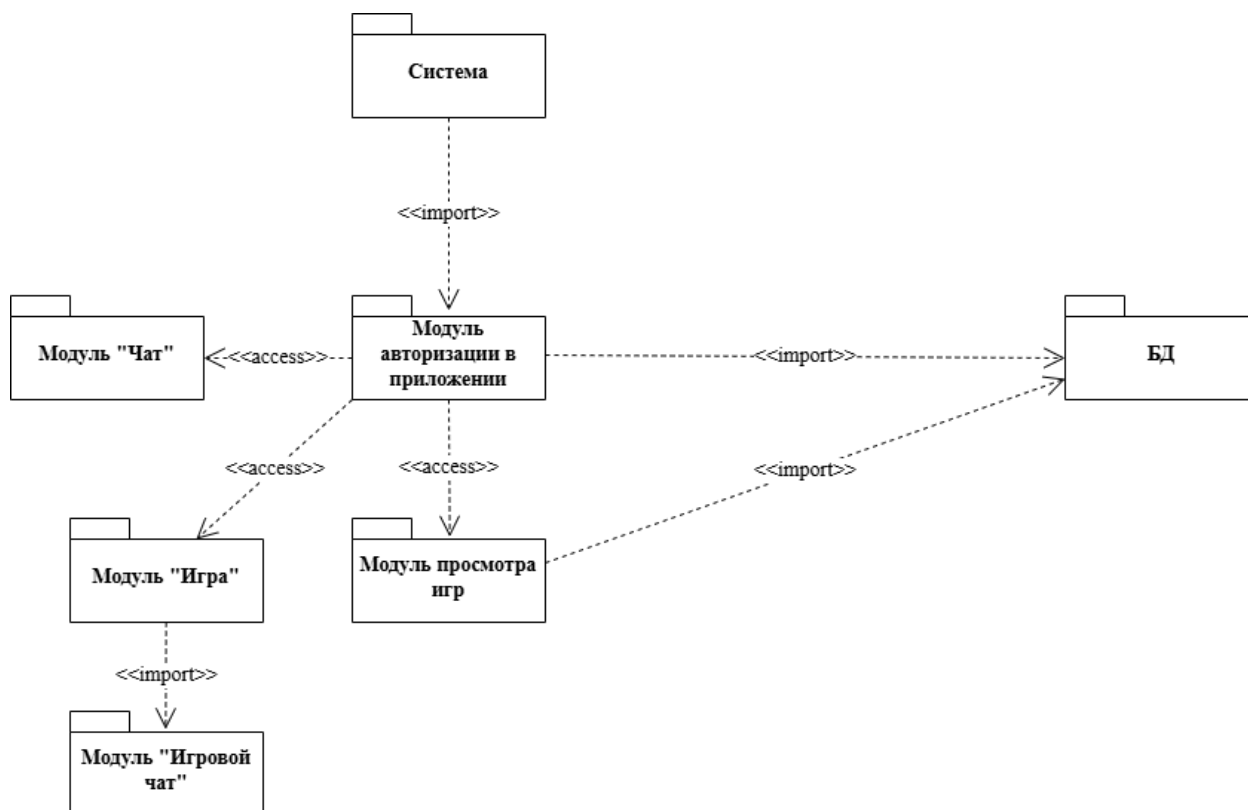


Рисунок 2.5 – Диаграмма пакетов

На данной диаграмме представлена структура приложения, разделённая на отдельные модули. Внешняя «Система» обозначает сторонние сервисы или API, с которыми может взаимодействовать приложение. Центральным элементом схемы является модуль авторизации в приложении – именно через него проходят все проверки доступа пользователей и вход в систему. Этот модуль взаимодействует с базой данных для хранения и получения информации о пользователях.

Внутри приложения выделены различные функциональные модули. Есть модуль чата, который обеспечивает обмен сообщениями между пользователями, и он также связан с модулем авторизации для проверки, кто именно пользуется чатом. Модуль игры реализует игровой процесс и может использовать отдельный игровой чат, встроенный в него как внутренний элемент. Ещё один модуль — это модуль просмотра игр, который позволяет пользователям просматривать доступные игры и напрямую работает как с модулем авторизации, так и с базой данных для получения нужных сведений.

Глава 3 Разработка мобильного приложения

3.1 Выбор технологий для разработки

Разработка мобильного приложения для шахмат является многогранной и достаточно трудоемкой задачей, требующей грамотного выбора технологического стека. Так как конечной целью является создание кроссплатформенного решения, особое внимание уделяется инструментам, которые обеспечивают удобство в реализации, надёжность и простоту масштабирования.

Учитывая, что основным языком программирования выбран Python, начальный этап анализа включал изучение его применимости для мобильной разработки. На сегодняшний день одним из самых популярных средств для создания мобильных приложений на языке Python является фреймворк Kivy. Он поддерживает кроссплатформенность, что позволяет создавать приложения для Android и iOS, не переписывая код под каждую платформу. Kivy предоставляет большое количество встроенных компонентов для построения пользовательского интерфейса, что особенно важно для реализации шахматной доски, фигур и игровых элементов.

Что касается серверной части, Python обладает широким выбором фреймворков для разработки web-сервисов, однако одним из наиболее подходящих вариантов является FastAPI. Этот инструмент отличает высокая производительность, асинхронная обработка запросов и простота интеграции с другими современными решениями. Для хранения и управления данными выбрана база данных PostgreSQL как надёжное и масштабируемое решение, идеально подходящее для работы с пользовательскими аккаунтами, результатами партий, историей ходов и другими важными сущностями.

Для обмена данными между клиентской и серверной частями рассматривается использование WebSocket, что обеспечит мгновенный отклик и синхронизацию игры между двумя устройствами. Фреймворк Kivy

имеет инструменты для работы с веб-сокетами, а FastAPI отлично поддерживает этот протокол.

Средой разработки выбран PyCharm – интегрированная среда, обладающая мощными возможностями для отладки и рефакторинга Python-кода, что ускоряет весь процесс создания качественного приложения.

Более детальное описание выбранных инструментов будет приведено в следующей части работы.

3.2 Описание выбранных средств для разработки мобильного приложения

Python(см. рисунок 3.1). Python – это динамически типизированный, интерпретируемый язык программирования, известный своей лаконичностью и простотой синтаксиса. Он универсален и активно используется в различных областях: от автоматизации и научных вычислений до искусственного интеллекта и веб-разработки. Для мобильной разработки Python не так популярен, как, например, Java или Kotlin, однако при правильном выборе сопутствующих технологий и фреймворков способен дать быстрый старт и обеспечить высокую продуктивность разработки. Сильными сторонами Python являются большое сообщество, огромное количество библиотек, простота освоения и возможность быстрой интеграции с другими языками и технологиями.



Рисунок 3.1 – Логотип языка программирования Python

Kivy. Для разработки пользовательского интерфейса мобильного клиента было решено использовать фреймворк Kivy. Это открытая библиотека для Python, предназначенная для создания кроссплатформенных приложений. Kivy поддерживает Windows, Linux, macOS, Android и iOS, что позволяет создавать универсальные приложения с единой кодовой базой. Инструментарий Kivy охватывает весь цикл построения графического интерфейса – от простых кнопок и текстовых полей до анимаций и сложных пользовательских элементов. Важно отметить, что фреймворк хорошо подходит для работы с графикой и имеет встроенные средства для обработки событий касания, масштабирования, перетаскивания объектов, что делает его особенно удобным при создании таких игр, как шахматы. Использование Kivy позволяет существенно ускорить процесс прототипирования и тестирования новых идей прямо на целевой платформе.

FastAPI(см. рисунок 3.2). Серверная часть приложения реализована с использованием FastAPI – современного высокопроизводительного веб-фреймворка для Python. FastAPI выделяется на фоне конкурентов (например, Flask или Django) встроенной поддержкой асинхронных операций, возможностью автоматической генерации документации (Swagger и OpenAPI), а также высокой скоростью работы, сравнимой с фреймворками на других языках. Благодаря FastAPI становится возможным реализовывать REST API и гибко управлять логикой обработки запросов от мобильного клиента, обеспечивая быстрое реагирование и высокую отказоустойчивость сервера. В рамках шахматного приложения сервер отвечает за аутентификацию пользователей, хранение и обработку данных партий, а также обеспечение связи между игроками.



Рисунок 3.2 – Логотип фреймворка FastAPI

PostgreSQL(см. рисунок 3.3). В качестве системы управления базами данных выбрана PostgreSQL. Эта объектно-реляционная СУБД хорошо зарекомендовала себя на рынке благодаря стабильности, высокой производительности и расширяемости. PostgreSQL поддерживает сложные типы данных, транзакции, индексы и предоставляет мощные возможности для обработки сложных запросов. В приложении для игры в шахматы PostgreSQL используется для хранения информации о пользователях, их результатах, ходе партий, статистике и прочих данных, необходимых для функционирования системы. Важно и то, что PostgreSQL благодаря открытости к расширениям позволяет при необходимости интегрировать новые функции без серьёзных архитектурных изменений.



Рисунок 3.3 – Логотип СУБД PostgreSQL

WebSocket. Для реализации онлайн-игры крайне важно обеспечить быструю и надёжную синхронизацию состояния между клиентами. Для этой задачи выбран протокол WebSocket, который обеспечивает постоянное двустороннее соединение между клиентом и сервером, в отличие от традиционных HTTP-запросов, работающих по принципу «запрос-ответ». С помощью WebSocket можно мгновенно передавать сообщения о ходах, состоянии доски, статусе соперника и других игровых событиях, что гарантирует максимальный отклик и высокую интерактивность игры. Благодаря поддержке WebSocket во FastAPI и наличию соответствующих библиотек для Python и Kivy, реализация такой архитектуры становится простой и понятной.

PyCharm(см. рисунок 3.4). В процессе разработки особое место занимает выбор среды программирования. В качестве основной IDE было выбрано PyCharm. Это профессиональная интегрированная среда, созданная для разработки на Python. Она поддерживает интеллектуальное автодополнение, мощные средства отладки и тестирования, а также возможности для управления зависимостями и виртуальными окружениями. PyCharm существенно ускоряет процесс поиска и устранения ошибок, повышает качество кода и удобство работы над средними и крупными проектами. Благодаря интеграции с системами контроля версий и поддержке различных плагинов, PyCharm становится незаменимым инструментом для командной и индивидуальной работы.

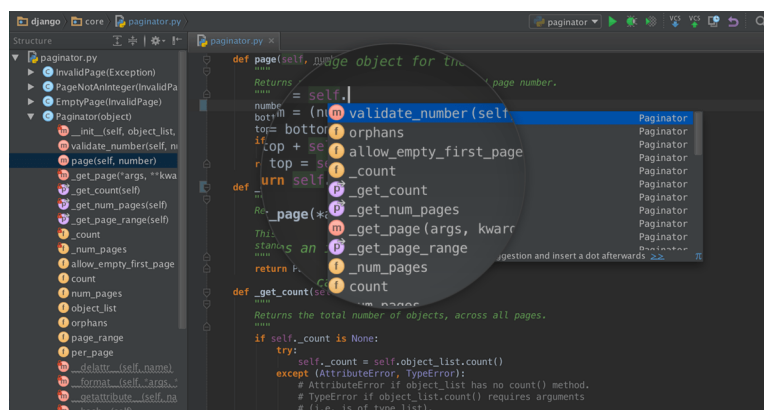


Рисунок 3.4 – Среда разработки PyCharm

3.3 Структура базы данных

Мобильное приложение представляет собой комплекс баз данных, включающий несколько основных таблиц, необходимых для функционирования приложения.

Таблица 1 – «Пользователь»

№	Наименование	Тип данных	Назначение
1	id	Int(11)auto_inc	Первичный ключ
2	Игровой никнэйм	Varchar(255)	Уникальное пользовательское имя
3	Количество побед	Int(11)	Количество побед в играх
4	Количество игр	Int(11)	Количество игр
5	Рейтинг	Int(11)	Рейтинг пользователя

Таблица №2 – «Партия»

№	Наименование	Тип данных	Назначение
1	id_партии	Int(11)auto_in с	Первичный ключ
2	id_первого_пользователя	Int(11)	Внешний ключ от таблицы “пользователь”
3	id_второго_пользователя	Int(11)	Внешний ключ от таблицы “пользователь”
4	Победитель	Int(11)	Внешний ключ от таблицы “Пользователь”
5	Длительность	Int(11)	Длительность партии
6	Количество ходов	Int(11)	Количество ходов
7	Дата проведения	DateTime	Дата проведения партии

Таблица №3 – «Достижение»

№	Наименование	Тип данных	Назначение
1	id_достижения	Int(11)auto_inc	Первичный ключ
2	id_пользователя	Int(11)	Внешний ключ от таблицы “Пользователь”
3	Название	Varchar(255)	Название достижения
4	Описание	Varchar(255)	Описание достижения
5	Дата получения	DateTime	Дата получения достижения

Таблица №4 – «Сообщение»

№	Наименование	Тип данных	Назначение
1	id_сообщения	Int(11)auto_inc	Первичный ключ
2	id_пользователя	Int(11)	Внешний ключ от таблицы “Пользователь”
3	текст сообщения	Varchar(255)	Текст сообщения

3.4 Структура мобильного приложения

Страница авторизации(см. рисунок 3.5). Когда пользователь заходит на сайт, перед ним сразу появляется страница входа. На этапе проектирования было принято решение ограничить доступ к сайту для неавторизованных пользователей. Поэтому, чтобы воспользоваться возможностями ресурса, посетителю нужно войти в свой аккаунт или зарегистрироваться.

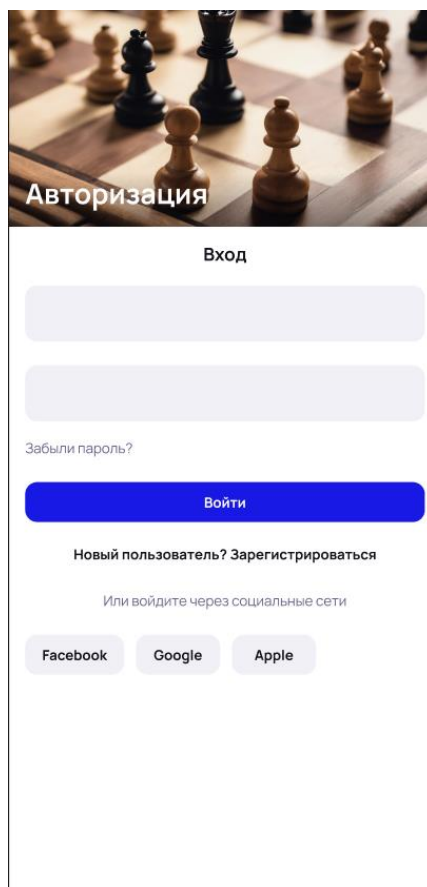


Рисунок 3.5 – Окно авторизации в приложении

На рисунке 3.5 представлена страница для входа в основную часть приложения. У пользователя также есть возможность перехода на страницу регистрации.

Страница регистрации(см. рисунок 3.6). На странице регистрации пользователь может создать новый аккаунт в мобильном приложении.

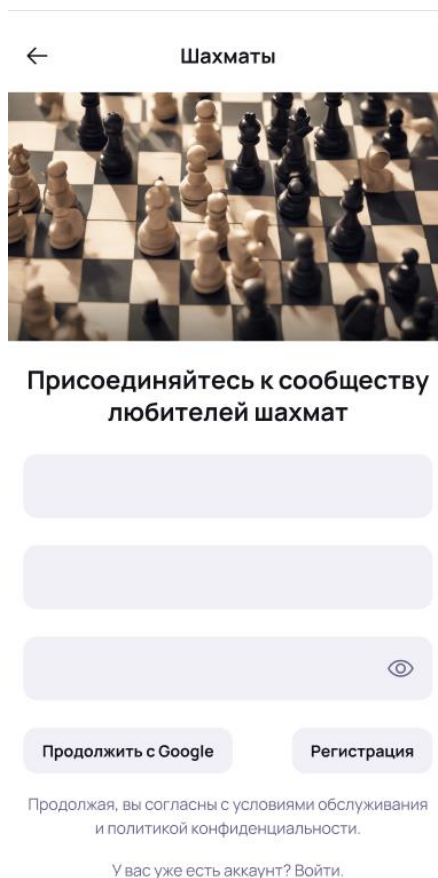


Рисунок 3.6 – Окно регистрации в приложении

На рисунке 3.6 показан экран, отображающий процесс создания нового пользователя. После ввода необходимых данных и подтверждения регистрации сервером пользователь будет автоматически перенаправлен на главную страницу игры.

Главная страница(см. рисунок 3.7). После завершения авторизации или регистрации пользователь перенаправляется на главную страницу мобильного приложения, которая предоставляет возможность посетить все основные разделы приложения.

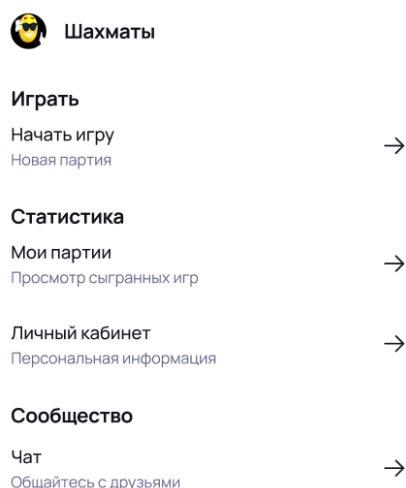


Рисунок 3.7 – Главная страница приложения

На представленной странице отображается главное меню мобильного приложения для игры в шахматы. В верхней части расположено название приложения и аватар игрока.

Основная часть интерфейса состоит из списка крупных интерактивных кнопок, каждая из которых отвечает за переход в соответствующий раздел приложения.

Страница «Чат»(см. рисунок 3.8). При нажатии по кнопке «Чат для общения» на главной странице, пользователь перенаправляется на страницу «Чат».

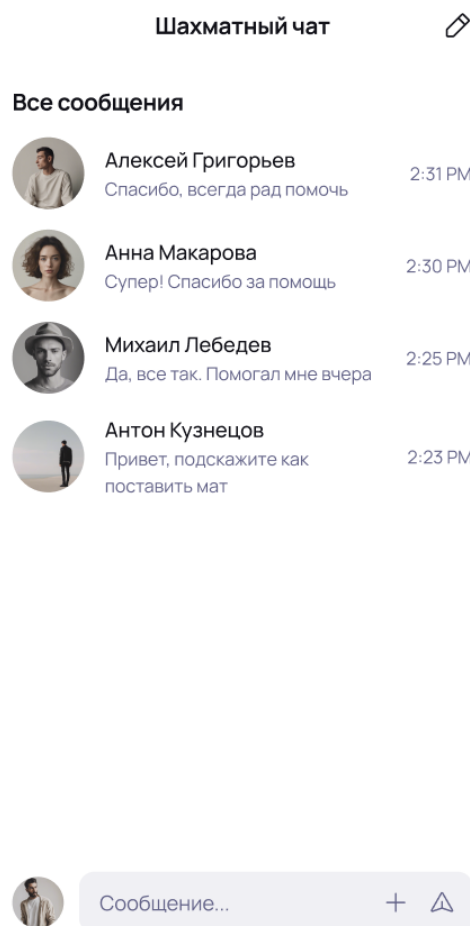


Рисунок 3.8 – Страница «Чат»

Страница чата для мобильного приложения шахматной игры выглядит современно и удобно для пользователя. В верхней части экрана расположен заголовок, отображающий название раздела или значок чата. Ниже находится основной экран, где отображаются сообщения от игроков. Каждое сообщение сопровождается никнеймом отправителя. В нижней части страницы располагается поле для ввода сообщения с кнопкой отправки, чтобы игроки могли быстро и просто обмениваться репликами во время или вне партии.

При нажатии на кнопку «Личный кабинет» на главной странице, пользователь попадет на страницу с его персональной информацией(см. рисунок 3.9).

Профиль



Anton Ivanov

@antonivanov

Зарегистрирован 10 месяцев назад

Статистика



20 Побед

1 Ничья



10 Поражений

1 Автоматическая победа

Достижения



Связанность

10 Побед в ряд



Чемпион

Победите 100 раз

Рисунок 3.9 – Страница личного кабинета пользователя

В верхней части отображается крупный аватар игрока, далее — его ник и текущий уровень, которые выделены шрифтом. Ниже следует раздел с достижениями: каждый трофей сопровождается тематическим значком и кратким описанием, список достижений можно прокручивать. В самом низу страницы находится удобная кнопка для перехода к настройкам профиля. Страница оптимизирована для мобильного устройства — все элементы достаточно крупные и отлично читаются, а сам интерфейс предоставляет игроку всю основную информацию о его успехах и статусе в шахматах.

«Список партий»(см. рисунок 3.10) — страница с информацией о текущих шахматных партиях.



Мои партии

	Виктория Победа	2 недели назад
	Алексей Поражение	3 недели назад
	Елизавета Ничья	4 недели назад
	Михаил Поражение	1 месяц назад

Рисунок 3.10 – Страница «Список партий»

На ней представлен список последних партий с указанием имени соперника, результата (Победа, Поражение или Ничья) и времени, прошедшего с момента окончания игры (например, "2 недели назад", "1 месяц назад"). Данные отсортированы в обратном хронологическом порядке, начиная с самых недавних партий вверху списка. Страница предоставляет пользователю быстрый обзор его игровой активности и результатов против различных оппонентов за последний месяц.

На рисунке 3.11 представлена страница «Смена аккаунта». Это интерфейс управления мультиаккаунтностью в приложении, предназначенный для переключения между разными пользовательскими профилями. Заголовок "Смена аккаунта" четко обозначает цель экрана. Под ним размещена краткая инструкция: "Переключитесь на другой аккаунт или добавьте новый", которая объясняет две ключевые функции раздела.

Основную часть экрана занимает список привязанных учетных записей, представленных в виде интерактивных карточек с радиокнопками для выбора. Каждый пункт содержит:

- тип аккаунта, показывающий метод аутентификации;
- логин пользователя.

Пустые чекбоксы указывают, что ни один из аккаунтов не выбран в данный момент – пользователь должен явно отметить нужный профиль для переключения. В нижней части экрана расположена выделенная кнопка "Добавить другой аккаунт", которая инициирует процесс привязки нового профиля (например, через социальные сети или email-регистрацию).

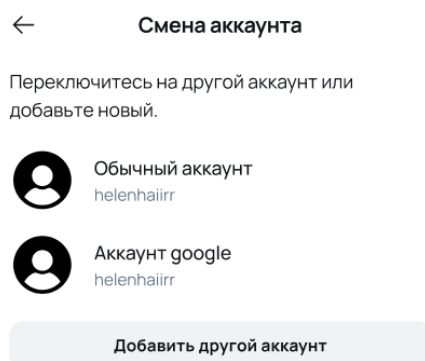


Рисунок 3.11 – Страница смены аккаунта

На рисунке 3.12 представлена страница, отображающая игровой процесс.



Рисунок 3.12 – Страница с игровым процессом

В верхней части отображается заголовок "Партия", информация о сопернике (никнейм VAN4SH) и таймер с общим временем игры (03:45). Под разделительной чертой расположены две ключевые кнопки управления: "Сдаться" для досрочного завершения партии и "Анализ ходов" для просмотра и разбора текущей позиции. Ниже условно представлено текстовое отображение шахматной доски в виде строк с буквенными координатами (a-h), что заменяет графическую визуализацию ходов. Интерфейс сфокусирован на минималистичном отображении текущего состояния игры и базовых действиях игрока.

Заключение

Выпускная квалификационная работа была посвящена разработке мобильного приложения для игры в шахматы, что является актуальной задачей в условиях стремительного роста популярности мобильных устройств и пользовательских приложений для интеллектуального досуга. В ходе выполнения работы последовательно решались задачи, обеспечивающие достижение поставленной цели:

- произведен детальный анализ существующих мобильных приложений для игры в шахматы, рассмотрены их основные функции, пользовательские сценарии, а также технологические особенности и недостатки;
- проведен обзор современных инструментальных средств и технологий для разработки мобильных приложений, что позволило выбрать оптимальные подходы и решения;
- выполнено объектно-ориентированное моделирование будущего приложения, определены ключевые сущности системы, их свойства и взаимосвязи между ними;
- реализовано мобильное приложение для игры в шахматы.

Теоретическая часть работы раскрывает специфику мобильной разработки и современные тенденции развития приложений для интеллектуальных игр. Проанализированы особенности реализации шахматных движений и методы организации игрового процесса в мобильных средах, что позволило грамотно адаптировать классические механики шахмат под формат мобильного устройства.

В значительной степени внимание было уделено вопросам построения архитектуры приложения с использованием принципов объектно-ориентированного программирования. Разработанная модель класса включает в себя объекты для представления шахматной доски, фигур, игроков, хода партии, а также механизмы, обеспечивающие корректное

выполнение правил игры и взаимодействие пользователя с системой. Такой подход гарантирует структурированность кода, гибкость в развитии приложения и возможность масштабирования функционала в будущем.

Результатом проведённой разработки стало создание полнофункционального мобильного приложения, позволяющего пользователям играть в шахматы на современных устройствах, просматривать историю партий, а также совершенствовать свои навыки и стратегическое мышление. Приложение обладает удобством использования, современным визуальным оформлением и высокой производительностью.

Практическая значимость работы заключается в возможности внедрения разработанного мобильного приложения в широкий образовательный и развлекательный контекст. Приложение может использоваться как начинающими, так и опытными шахматистами для регулярной практики или участия в онлайн-соперничестве с другими пользователями. Оно также может послужить платформой для дальнейших исследований и внедрения новых функций, таких как организация онлайн-турниров, интеграция с шахматными клубами или искусственным интеллектом различного уровня сложности.

В целом, выполненное исследование и разработка позволяют сделать вывод о целесообразности применения современных технологий и принципов объектно-ориентированного программирования при создании мобильных приложений для интеллектуальных игр. Реализация поставленных задач позволила не только обогатить теоретические и практические знания в сфере мобильной разработки, но и создать качественный продукт, который отвечает современным требованиям пользователей, способствует популяризации шахмат и развитию логического мышления.

Список использованной литературы

1. Аль Мусави О.А.Р., Кравец О.Я. Исследование алгоритмов повторной оптимизации запросов в облачных базах данных // Решение. 2022. Т. 1. С. 168-171.
2. Амалбеков С.С., Тусупов Д.А. Использование инструмента dbms_job в oracle для планирования и управления заданиями в базе данных // Интернаука. 2023. № 19-1 (289). С. 49-51.
3. Безруков И.А., Сальников А.И., Яковлев В.А., Вылегжанин А.В. Анализ надежности программного отказоустойчивого массива при организации системы долговременного хранения данных // Приборы и техника эксперимента. 2022. № 2. С. 37-42.
4. Бирих Э.В., Волокушина К.П. Разработка автоматизированного процесса по корреляции информации между таблицей базы данных PostgreSQL и Active Directory // Экономика и качество систем связи. 2023. № 1 (27). С. 65-71.
5. Бородина Е.А., Даценко Н.В., Никитин Б.Е., Мачтаков С.Г., Хромых Е.А. Проектирование баз данных. Учебное пособие для подготовки обучающихся по направлениям 09.03.02 - «Информационные системы и технологии», 09.03.03 «Прикладная информатика» / Воронеж, 2023.
6. Варакута П.С., Козлов Р.К. Имитационное моделирование пропускной способности пулов соединений к базе данных POSTGRESQL // Трибуна ученого. 2022. № 5. С. 48-53.
7. Волк В.К. Базы данных. Проектирование, программирование, управление и администрирование. Учебник для вузов. Сер. Высшее образование. Санкт-Петербург, 2022.
8. Ганин Д., Маслова Е. Вопросы унификации терминов различных баз данных по товарам и услугам // Интеллектуальная собственность. Промышленная собственность. 2022. № 7. С. 49-53.

9. Герасимов Б.Н. Обновление подпроцесса управления базой данных по персоналу организации // Экономика и бизнес: теория и практика. 2022. № 2 (84). С. 38-43.
10. Гранкин В.Е. Система управления базами данных OPENOFFICE BASE. Практикум. Москва, 2022.
11. Денисенко С.А., Колобаев В.А., Матвеев К.В., Скотаренко Е.Ю. Создание единой цифровой базы данных по свойствам веществ и материалов на основе стандартных справочных данных // Главный метролог. 2022. № 1 (124). С. 10-15.
12. Жук М.М. SQL: от традиционных баз данных к большим данным // Точная наука. 2022. № 136. С. 4-8.
13. Зубков А. Е. Информатика и языки программирования : практикум / А. Е. Зубков, Е. В. Зубкова, Т. В. Кортеева. - Екатеринбург : Изд-во УрГЭУ, 2020. - 98 с.
14. Иванова А.К., Куленцан А.Л. Базы данных. Системы управления базами данных // Сборник научных трудов вузов России "Проблемы экономики, финансов и управления производством". 2023. № 52. С. 110-114.
15. Ильясов А., Чарыева Г. База данных и требования к базе данных // Ceteris Paribus. 2022. № 12. С. 28-30.
16. Информатика и информационные технологии : учеб. пособие / С. В. Бегичева, Е. В. Долженкова, И. Е. Жуковская [и др.] ; под общ. ред. Д. М. Назарова. - Екатеринбург : Изд-во УрГЭУ, 2020. - 219 с.
17. Кислицын Е. В. Инструменты обработки и анализа корпоративных данных : учеб. пособие / Е. В. Кислицын, Е. М. Кочкина, Е. В. Радковская. - Екатеринбург : Изд-во УрГЭУ, 2021. - 227 с.
18. Кислицын Е. В. Современные технологии разработки программного обеспечения : учеб. пособие / Е. В. Кислицын, М. А. Панов. - Екатеринбург : Изд-во УрГЭУ, 2021. - 176 с.
19. Минина Е. Е. Распределенные системы и облачные технологии : учеб. пособие / Е. Е. Минина. - Екатеринбург : Изд-во УрГЭУ, 2020. - 122 с.

20. Морякова Ю.М. Базы данных как инструмент обобщения социально-экономических данных // Вестник Владимирского государственного университета имени Александра Григорьевича и Николая Григорьевича Столетовых. Серия: Экономические науки. 2023. № 3 (37). С. 115-119.

21. Официальный учебный курс Microsoft. Microsoft Office Access 2003 (+ CD-ROM). - М.: Эком, Бинوم. Лаборатория знаний, 2022. - 277 с.

22. Полтавцева М.А. Безопасность баз данных: учебное пособие для студентов высших учебных заведений, обучающихся по УГСН 10.00.00 «Информационная безопасность» по программам подготовки бакалавров, магистров, специалистов. Санкт-Петербург, 2023.

23. Полтавцева М.А. Безопасность поли-баз данных в системах управления большими данными // Методы и технические средства обеспечения безопасности информации. 2023. № 32. С. 40-41.

24. Потапов И.И., Камнева И.Е., Журавлева И.П. Анализ тематических фрагментов баз данных ВИНТИ РАН по проблемам устойчивого развития // Проблемы окружающей среды и природных ресурсов. 2023. № 8. С. 48-57.

25. Преппернау, Дж. Microsoft Office Access 2007. Русская версия (+ CD-ROM) / Дж. Преппернау. - М.: Эком, 2022. - 382 с.

26. Приходько С.В. Теоретические основы и общие принципы построения современных баз данных и систем управления базами данных // Матрица научного познания. 2023. № 4-1. С. 128-131.

27. Романьков В.А. Обобщенная схема скрытого компактного хранения данных различных пользователей в общей открытой базе // Известия Иркутского государственного университета. Серия: Математика. 2022. Т. 40. С. 63-77.

28. Санданов Д.В. Особенности работы с базами данных по распространению растений и опыт консолидации данных различного формата // Природа Внутренней Азии. 2022. № 4 (22). С. 96-104.

29. Смирнова, О. В. Access 2007 на практике / О.В. Смирнова. - М.: Феникс, 2019. - 339 с.

30. Обзор возможностей фреймворка Kivy и библиотеки KivyMD // Хабр URL: <https://habr.com/ru/articles/546684/> (дата обращения: 14.03.2025).

31. Kivy // SkillFactory URL: <https://blog.skillfactory.ru/glossary/kivy/> (дата обращения: 21.03.2025).

32. Путеводитель по Docker. От основ контейнеризации до создания собственного докера // Хабр URL: <https://habr.com/ru/articles/810777/> (дата обращения: 02.04.2025).

33. Публикуем Android приложения автоматически с помощью Gitlab CI // Хабр URL: <https://habr.com/ru/companies/gazprommedia/articles/753984/> (дата обращения: 07.04.2025).

```

from kivy.app import App
from kivy.uix.screenmanager import Screen, ScreenManager
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.core.window import Window
from kivy.graphics import Color, Rectangle, Line, RoundedRectangle
from kivy.metrics import dp
from kivy.properties import ListProperty, StringProperty
from kivy.lang import Builder

# Цвета кнопок (R, G, B)
Window.clearcolor = (0.95, 0.95, 0.95, 1)

class SocialButton(Button):
    bg_color = ListProperty([0.2, 0.4, 0.8, 1])
    radius = ListProperty([dp(10)] * 4)

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.size_hint = (None, None)
        self.size = (dp(80), dp(50))
        self.color = (1, 1, 1)
        self.font_size = dp(14)
        self.bold = True
        self.bind(pos=self.update_canvas, size=self.update_canvas)

    def update_canvas(self, *args):
        self.canvas.before.clear()
        with self.canvas.before:
            Color(self.bg_color)
            RoundedRectangle(pos=self.pos, size=self.size, radius=self.radius)

class CustomLabel(Label):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.color = (0.3, 0.3, 0.3, 1)
        self.font_size = dp(16)

class HeaderLabel(Label):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.color = (0.1, 0.1, 0.1, 1)
        self.bold = True
        self.font_size = dp(32)
        self.size_hint_y = None
        self.height = dp(50)

class SubHeaderLabel(Label):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.color = (0.3, 0.3, 0.3, 1)
        self.font_size = dp(24)
        self.size_hint_y = None
        self.height = dp(40)

class TextButton(Button):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.background_color = (0, 0, 0, 0)
        self.color = (0.2, 0.4, 0.8, 1)
        self.font_size = dp(16)
        self.size_hint_y = None
        self.height = dp(30)

class MainButton(Button):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.background_color = (0.2, 0.4, 0.8, 1)
        self.color = (1, 1, 1)
        self.font_size = dp(18)
        self.bold = True
        self.size_hint_y = None
        self.height = dp(50)

class Divider(Widget):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.size_hint_y = None
        self.height = dp(1)
        self.bind(pos=self.update_canvas, size=self.update_canvas)

    def update_canvas(self, *args):
        self.canvas.clear()
        with self.canvas:
            Color(0.8, 0.8, 0.8, 1)
            Line(points=[self.x, self.center_y, self.right, self.center_y], width=1)

class AuthScreen(Screen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.build_ui()

    def build_ui(self):
        main_layout = BoxLayout(
            orientation='vertical',
            padding=[dp(40), dp(20), dp(40), dp(40)],
            spacing=dp(20)
        )

        # Заголовок
        main_layout.add_widget(HeaderLabel(text='Авторизация'))

        # Подзаголовок
        main_layout.add_widget(SubHeaderLabel(text='Вход'))

        # Ссылка «Забыли пароль?»
        forgot_btn = TextButton(text='Забыли пароль?')
        forgot_btn.bind(on_press=self.forgot_password)
        main_layout.add_widget(forgot_btn)

        # Кнопка «Вход»
        login_btn = MainButton(text='Войти')
        login_btn.bind(on_press=self.login)
        main_layout.add_widget(login_btn)

        # Регистрация
        reg_layout = BoxLayout(
            size_hint_y=None,
            height=dp(40),
            spacing=dp(5)
        )
        reg_layout.add_widget(CustomLabel(text='Новый пользователь?'))
        reg_btn = TextButton(text='Зарегистрироваться')
        reg_btn.bind(on_press=self.register)
        reg_layout.add_widget(reg_btn)
        main_layout.add_widget(reg_layout)

        # Разделитель
        main_layout.add_widget(Divider())

        # Социальные сети
        main_layout.add_widget(CustomLabel(
            text='или войдите через социальные сети',
            size_hint_y=None,
            height=dp(40)
        ))

        # Кнопки соцсетей
        social_layout = BoxLayout(
            size_hint_y=None,
            height=dp(60),
            spacing=dp(30),
            padding=[dp(60), 0, dp(60), 0]
        )

        fb_btn = SocialButton(text='Facebook', bg_color=(0.23, 0.35, 0.58, 1))
        fb_btn.bind(on_press=self.fb_login)
        social_layout.add_widget(fb_btn)

        google_btn = SocialButton(text='Google', bg_color=(0.86, 0.26, 0.22, 1))
        google_btn.add_widget(google_btn)
        social_layout.add_widget(google_btn)

        apple_btn = SocialButton(text='Apple', bg_color=(0.1, 0.1, 0.1, 1))
        apple_btn.bind(on_press=self.apple_login)
        social_layout.add_widget(apple_btn)

        main_layout.add_widget(social_layout)
        self.add_widget(main_layout)

    def login(self, instance):
        print(f'Выполняется вход с помощью: {instance.text}')

    def register(self, instance):
        print(f'Переход к регистрации: {instance.text}')

    def forgot_password(self, instance):
        print(f'Восстановление пароля: {instance.text}')

    def fb_login(self, instance):
        print(f'Вход через Facebook: {instance.text}')

    def google_login(self, instance):
        print(f'Вход через Google: {instance.text}')

    def apple_login(self, instance):
        print(f'Вход через Apple: {instance.text}')

class AuthApp(App):
    def build(self):
        sm = ScreenManager()
        sm.add_widget(AuthScreen(name='auth'))
        return sm

if __name__ == '__main__':
    AuthApp().run()

```



```

from kivy.app import App
from kivy.uix.screenmanager import Screen, ScreenManager
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.gridlayout import GridLayout
from kivy.uix.button import Button
from kivy.uix.label import Label
from kivy.uix.image import Image
from kivy.core.window import Window
from kivy.metrics import dp
from kivy.graphics import Color, Rectangle, RoundedRectangle
from kivy.properties import ListProperty, StringProperty, NumericProperty
from kivy.lang import Builder

# Установка цвета окна
Window.clearcolor = (0.96, 0.91, 0.81, 1) # Кремовый фон, как шахматная доска

Builder.load_string('''
-ChessButtons:
    background_color: (0, 0, 0, 0)
    canvas.before:
        Color:
            rgba: self.bg_color
        RoundedRectangle:
            pos: self.pos
            size: self.size
            radius: [dp(10),]

# Темная рамка
    Line:
        rounded_rectangle: [self.x, self.y, self.width, self.height, dp(10)]
        width: dp(1.5)

BoxLayout:
    orientation: 'vertical'
    padding: [dp(15), dp(10)]
    spacing: dp(5)

    Label:
        text: root.main_text
        font_size: root.main_font_size
        bold: True
        color: root.text_color
        text_size: self.width, None
        halign: 'center'
        valign: 'middle'
        height: self.texture_size[1]
        size_hint_y: None

    Label:
        text: root.sub_text
        font_size: root.sub_font_size
        color: root.text_color
        text_size: self.width, None
        halign: 'center'
        valign: 'top'
        height: self.texture_size[1]
        size_hint_y: None
...

class ChessButton(Button):
    bg_color = ListProperty([0.76, 0.65, 0.42, 1]) # Цвет дерева шахматной доски
    text_color = ListProperty([0.1, 0.1, 0.1, 1]) # Темная темка
    sub_text = StringProperty('')
    main_font_size = NumericProperty(dp(24))
    sub_font_size = NumericProperty(dp(16))

class ChessScreen(Screen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.build_ui()

    def build_ui(self):
        # Основная часть экрана
        main_layout = BoxLayout(
            orientation='vertical',
            padding=[dp(20), dp(20), dp(20), dp(10)],
            spacing=dp(25)
        )

        # Заголовок
        header = BoxLayout(size_hint_y=None, height=dp(80))
        with header.canvas.before:
            Color([0.4, 0.2, 0.1, 1]) # Темно-коричневый
            Rectangle(pos=header.pos, size=header.size)

        title_label = Label(
            text='Шахматы',
            font_size=dp(42),
            bold=True,
            color=(1, 0.9, 0.7, 1),
            size_hint_x=0.8
        )

        # Иконка шахмат
        chess_icon = Image(
            source='', # Здесь можно указать путь к иконке
            size_hint=(None, None),
            size=(dp(60), dp(60)),
            color=(1, 0.5, 0.7, 1)
        )

        header.add_widget(title_label)
        header.add_widget(chess_icon)
        main_layout.add_widget(header)

        # Сетка кнопок
        grid = GridLayout(
            cols=2,
            spacing=dp(20),
            padding=[dp(10), 0, dp(10), dp(10)],
            size_hint_y=0.8
        )

        # Секция 1: Игры
        play_btn = ChessButton(
            main_text='Игры',
            sub_text='Начать новую игру',
            bg_color=(0.86, 0.75, 0.55, 1)
        )
        play_btn.bind(on_press=self.play)
        grid.add_widget(play_btn)

        # Секция 2: Статистика
        stats_btn = ChessButton(
            main_text='Статистика',
            sub_text='Моя статистика и просмотр сыгранных игр',
            bg_color=(0.76, 0.65, 0.42, 1)
        )
        stats_btn.bind(on_press=self.show_stats)
        grid.add_widget(stats_btn)

        # Секция 3: Личная кабинет
        profile_btn = ChessButton(
            main_text='Личный кабинет',
            sub_text='Персональная информация',
            bg_color=(0.86, 0.75, 0.55, 1)
        )
        profile_btn.bind(on_press=self.open_profile)
        grid.add_widget(profile_btn)

        # Секция 4: Сообщество
        community_btn = ChessButton(
            main_text='Сообщество',
            sub_text='Чат и обмен опытом с друзьями',
            bg_color=(0.76, 0.65, 0.42, 1)
        )
        community_btn.bind(on_press=self.open_community)
        grid.add_widget(community_btn)
        main_layout.add_widget(grid)

        # Нижняя панель
        footer = BoxLayout(size_hint_y=None, height=dp(50))
        with footer.canvas.before:
            Color([0.4, 0.2, 0.1, 1]) # Темно-коричневый
            Rectangle(pos=footer.pos, size=footer.size)

        copyright_label = Label(
            text='© Шахматы Клуб 2023',
            font_size=dp(16),
            color=(1, 0.9, 0.7, 1)
        )
        footer.add_widget(copyright_label)
        main_layout.add_widget(footer)

        self.add_widget(main_layout)

    def play(self, instance):
        print(f"Начало игры: {instance.main_text}")

    def show_stats(self, instance):
        print(f"Просмотр статистики: {instance.main_text}")

    def open_profile(self, instance):
        print(f"Открытие профиля: {instance.main_text}")

    def open_community(self, instance):
        print(f"Открытие сообщества: {instance.main_text}")

class ChessApp(App):
    def build(self):
        sm = ScreenManager()
        sm.add_widget(ChessScreen(name='main'))
        return sm

    def on_start(self):
        # Установка минимального размера окна как у мобильного устройства
        Window.size = (360, 640)

if __name__ == '__main__':
    ChessApp().run()

```

Код страницы «Чат»

58

Код страницы личного кабинета пользователя

[illegible]

```

from kivy.app import App
from kivy.uix.screenmanager import Screen, ScreenManager
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.scrollview import ScrollView
from kivy.uix.label import Label
from kivy.uix.widget import Widget
from kivy.core.window import Window
from kivy.metrics import dp
from kivy.graphics import Color, Rectangle, Line
from kivy.properties import ListProperty, StringProperty
from kivy.lang import Builder

# Устанавливаем ширину окна
Window.clearcolor = (0.96, 0.91, 0.81, 1) # Крановый фон

Builder.load_string('''
<GameResultCard>:
    orientation: 'vertical'
    padding: [dp(20), dp(15)]
    spacing: dp(8)
    size_hint_y: None
    height: dp(128)
    canvas.before:
        Color:
            rgba: root.bg_color
        RoundedRectangle:
            pos: self.pos
            size: self.size
            radius: [dp(12),]
        Color:
            rgba: 0.3, 0.2, 0.1, 0.5 # Темно-коричневая рамка
        Line:
            rounded_rectangle: [self.x, self.y, self.width, self.height, dp(12)]
            width: dp(1.2)

<ResultLabel>:
    font_size: dp(18)
    color: root.text_color
    bold: True
    size_hint_y: None
    height: dp(25)
    text_size: (self.width, None)
    halign: 'left'
    valign: 'middle'
...

class GameResultCard(BoxLayout):
    bg_color = ListProperty([0.86, 0.75, 0.55, 1]) # Светло-коричневый

class ResultLabel(Label):
    text_color = ListProperty([0.2, 0.2, 0.2, 1])

class GamesScreen(Screen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.games = [
            {'opponent': 'Виктория', 'result': 'Победа', 'time': '2 недели назад', 'color': (0.2, 0.5,
            0.2, 1)},
            {'opponent': 'Алексей', 'result': 'Поражение', 'time': '3 недели назад', 'color': (0.7,
            0.2, 0.2, 1)},
            {'opponent': 'Елизавета', 'result': 'Ничья', 'time': '4 недели назад', 'color': (0.8, 0.6,
            0.2, 1)},
            {'opponent': 'Михаил', 'result': 'Поражение', 'time': '1 месяц назад', 'color': (0.7, 0.2,
            0.2, 1)},
        ]
        self.build_ui()

    def build_ui(self):
        # Основной контейнер
        main_layout = BoxLayout(orientation='vertical')

        # Заголовок
        header = BoxLayout(
            size_hint_y=None,
            height=dp(60),
            padding=[dp(20), dp(10)],
        )
        with header.canvas.before:
            Color(0.3, 0.2, 0.1, 1) # Темно-коричневый
            Rectangle(pos=header.pos, size=header.size)

        header.add_widget(Label(
            text="Мои партии",
            font_size=dp(24),
            bold=True,
            color=(1, 0.9, 0.7, 1)
        ))
        main_layout.add_widget(header)

        # Прокручиваемая область
        scroll_view = ScrollView()
        content_layout = BoxLayout(
            orientation='vertical',
            spacing=dp(20),
            padding=[dp(20), dp(20), dp(20), dp(20)],
            size_hint_y=None
        )
        content_layout.bind(minimum_height=content_layout.setter('height'))

        # Добавляем карточки с результатами партий
        for game in self.games:
            card = GameResultCard()

            # Имя соперника
            opponent_label = ResultLabel(
                text=game['opponent'],
                font_size=dp(24),
                bold=True,
                text_color=(0.2, 0.2, 0.2, 1)
            )
            card.add_widget(opponent_label)

            # Результат
            result_label = ResultLabel(
                text=game['result'],
                text_color=game['color']
            )
            card.add_widget(result_label)

            # Время
            time_label = ResultLabel(
                text=game['time'],
                font_size=dp(16),
                bold=False,
                text_color=(0.5, 0.5, 0.5, 1)
            )
            card.add_widget(time_label)

            content_layout.add_widget(card)

        scroll_view.add_widget(content_layout)
        main_layout.add_widget(scroll_view)

        self.add_widget(main_layout)

class GamesApp(App):
    def build(self):
        sm = ScreenManager()
        sm.add_widget(GamesScreen(name='games'))
        return sm

    def on_start(self):
        # Устанавливаем размер окна как у мобильного устройства
        Window.size = (360, 640)

if __name__ == '__main__':
    GamesApp().run()

```

```

from kivy.app import App
from kivy.uix.screenmanager import Screen, ScreenManager
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.uix.togglebutton import ToggleButton
from kivy.uix.widget import Widget
from kivy.graphics import Image
from kivy.metrics import dp
from kivy.properties import Color, Rectangle, Line, Ellipse
from kivy.props import BooleanProperty, ListProperty
from kivy.lang import Builder

# Задаем размер окна
window.clearcolor = (0.95, 0.91, 0.81, 1) # Крестовый фон

Builder.load_string('''
<AccountOption>:
    orientation: 'horizontal'
    padding: [dp(15), dp(10)]
    spacing: dp(15)
    size_hint: None
    height: dp(70)
    canvas.before:
        Color: rgba: 0.85, 0.75, 0.55, 1 # Черно-коричневый
        RoundedRectangle:
            pos: self.pos
            size: self.size
            radius: [dp(10), 1]
        Color: rgba: 0.9, 0.2, 0.1, 0.5 # Темно-коричневая панель
        Line:
            rounded_rectangle: [self.x, self.y, self.width, self.height, dp(10)]
            width: dp(1.2)
...
class AccountOption(BoxLayout):
    pass
class RadioButton(Widget):
    active = BooleanProperty(False)
    color = ListProperty([0.9, 0.2, 0.1, 1]) # Темно-коричневая
    def __init__(self, *args):
        super().__init__(*args)
        self.size_hint = (None, None)
        self.size = (dp(20), dp(20))
        self.radius = self.size/2
        self.update_canvas()
    def update_canvas(self, *args):
        self.canvas.clear()
        with self.canvas:
            Color(self.color)
            Ellipse:
                pos=self.pos
                size=self.size
            # Рисуем панель (темно-коричневая)
            if self.active:
                Color: rgba: 0.2, 0.1, 1, 1 # Темно-коричневая
                Ellipse:
                    pos=(self.pos[0] + dp(7), self.pos[1] + dp(7)),
                    size=(dp(10), dp(10))
    def on_touch_down(self, touch):
        if self.collide_point(touch.x, touch.y):
            self.active = not self.active
            return True
        return super().on_touch_down(touch)
class SwitchAccountScreen(Screen):
    def __init__(self, *args):
        super().__init__(*args)
        self.radio_buttons = []
        self.build_ui()
    def build_ui(self):
        # Рисуем
        main_layout = BoxLayout(orientation='vertical')
        # Header
        header = BoxLayout()
        size = dp(100)
        height=dp(50)
        padding=[dp(5), dp(10)]
        with header.canvas.before:
            Color: rgba: 0.2, 0.1, 1, 1 # Темно-коричневая
            Rectangle:
                pos=header.pos
                size=header.size
        header.add_widget(Label(
            text="Добавить новый аккаунт",
            font_size=dp(24),
            bold=True,
            color=(1, 0.9, 0.7, 1)
        ))
        main_layout.add_widget(header)
        # ScrollView
        scroll_view = ScrollView()
        content_layout = BoxLayout(
            orientation='vertical',
            spacing=dp(20),
            padding=[dp(20), dp(20), dp(20), dp(20)]
        )
        scroll_view.add_widget(content_layout)
        content_layout.bind(minimum_height=content_layout.setter('height'))
        # Описание
        desc_label = Label(
            text="Добавьте аккаунт на любой аккаунт или добавьте новый.",
            font_size=dp(18),
            color=(1, 0.9, 0.7, 1),
            size_hint=(None, None),
            height=dp(20),
            width=dp(500)
        )
        content_layout.add_widget(desc_label)
        # Аккаунт 1
        account1 = AccountOption()
        # Рисуем панель
        radio1 = RadioButton()
        radio1.active = True # Аккаунт выбран по умолчанию
        self.radio_buttons.append(radio1)
        account1.add_widget(radio1)
        # Информация
        account_info1 = BoxLayout(orientation='vertical')
        account_info1.add_widget(Label(
            text="Добавить аккаунт",
            font_size=dp(20),
            bold=True,
            color=(0.2, 0.2, 0.2, 1),
            height=dp(20),
            size_hint=(None, None),
            height=dp(20)
        ))
        account_info1.add_widget(Label(
            text="Полное имя",
            font_size=dp(18),
            color=(1, 0.9, 0.7, 1),
            height=dp(20),
            size_hint=(None, None),
            height=dp(20)
        ))
        account1.add_widget(account_info1)
        content_layout.add_widget(account1)
        # Аккаунт 2
        account2 = AccountOption()
        # Рисуем панель
        radio2 = RadioButton()
        self.radio_buttons.append(radio2)
        account2.add_widget(radio2)
        # Информация
        account_info2 = BoxLayout(orientation='vertical')
        account_info2.add_widget(Label(
            text="Добавить аккаунт",
            font_size=dp(20),
            bold=True,
            color=(0.2, 0.2, 0.2, 1),
            height=dp(20),
            size_hint=(None, None),
            height=dp(20)
        ))
        account_info2.add_widget(Label(
            text="Полное имя",
            font_size=dp(18),
            color=(1, 0.9, 0.7, 1),
            height=dp(20),
            size_hint=(None, None),
            height=dp(20)
        ))
        account2.add_widget(account_info2)
        content_layout.add_widget(account2)
        # Добавить аккаунт
        add_account_btn = Button(
            text="Добавить новый аккаунт",
            background_color=(0.9, 0.2, 0.1, 1), # Темно-коричневая
            color=(1, 0.9, 0.7, 1),
            font_size=dp(20),
            bold=True,
            size_hint=(None, None),
            height=dp(30)
        )
        add_account_btn.bind(on_press=self.add_account)
        content_layout.add_widget(add_account_btn)
        scroll_view.add_widget(content_layout)
        main_layout.add_widget(scroll_view)
        self.add_widget(main_layout)
    def add_account(self, instance):
        print("Добавление нового аккаунта...")
    def on_touch_down(self, touch):
        # Проверяем, на какой радиобutton нажали
        for radio in self.radio_buttons:
            if radio.collide_point(touch.x, touch.y):
                # Делаем этот активным
                radio.active = True
                # Делаем остальные неактивными
                for r in self.radio_buttons:
                    r.active = False
                # Рисуем панель
                radio.update_canvas()
                print("Выбран аккаунт: {}".format(self.get_selected_account(radio)))
                return True
        return super().on_touch_down(touch)
    def get_selected_account(self, radio):
        index = self.radio_buttons.index(radio)
        if index == 0:
            return "Добавить аккаунт: helenhair"
        elif index == 1:
            return "Добавить аккаунт: helenhair"
        return "Добавить аккаунт: helenhair"
class SwitchAccountApp(App):
    def build(self):
        sm = ScreenManager()
        sm.add_widget(SwitchAccountScreen(name="switch_account"))
        return sm
    def on_start(self):
        # Задаем размер окна
        window.size = (360, 640)
        if __name__ == '__main__':
            SwitchAccountApp().run()

```



```

from kivy.app import App
from kivy.uix.screenmanager import Screen, ScreenManager
from kivy.uix.widget import Widget
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.uix.widget import Widget
from kivy.graphics import Color, Rectangle, Line
from kivy.properties import StringProperty, ListProperty
from kivy.lang import Builder
from kivy.clock import Clock

# Константы
WINDOW_WIDTH = 400
WINDOW_HEIGHT = 600
FPS = 60

# Инициализация
Builder.load_string("""
    Window.clearcolor = (0.50, 0.50, 0.50, 1) # Темно-серый фон

    # Шапка
    Builder.load_string("""
        font_size: dp(18)
        text: "Game of Thrones"
        text_size: self.width, None
        align: "left"
        valign: "middle"
    """)

    # Чекбокс
    checkbox = Checkbox()
    checkbox.bind(
        on_check=self.toggle_theme,
        on_uncheck=self.toggle_theme
    )

    # Информация о игре
    class GameInfoLabel(Label):
        pass

    class ChessBoard(GridLayout):
        pass

    class ChessSquare(Button):
        def __init__(self, color, **kwargs):
            super().__init__(**kwargs)
            self.background_color = color
            self.size_hint_x = 0.5
            self.size_hint_y = 0.5
            self.text = ""
            self.color = (0.5, 0.5, 0.5, 1) if color == (0.50, 0.50, 0.50, 1) else (1, 1, 1, 1)

    class GameScreen(Screen):
        game_time = StringProperty("03:45")

        def __init__(self, **kwargs):
            super().__init__(**kwargs)
            self.timer_running = False
            self.canvas = Canvas()
            self.canvas.clear()

        def build(self):
            # Основной контейнер
            main_layout = BoxLayout(orientation='vertical', padding=[dp(10), dp(10), spacing=dp(10)])

            # Шапка
            header = BoxLayout(
                size_hint_y=None,
                height=dp(200),
                padding=[dp(20), dp(10)],
            )
            with header.canvas.before():
                Color((0.5, 0.5, 0.5, 1)) # Темно-серый фон
                Rectangle(pos=header.pos, size=header.size)
            header.add_widget(Label(
                text="Game of Thrones",
                font_size=dp(20),
                bold=True,
                color=(1, 0.5, 0.5, 1)
            ))
            main_layout.add_widget(header)

            # Информация о игре
            info_layout = BoxLayout(orientation='vertical', spacing=dp(10), padding=[dp(20), 0])

            # Оппонент
            opponent_layout = BoxLayout(size_hint_y=None, height=dp(40))
            opponent_layout.add_widget(GameInfoLabel(text="Opponent"))
            opponent_layout.add_widget(Label(
                text="Opponent Name",
                font_size=dp(12),
                bold=True,
                color=(0.5, 0.5, 0.5, 1)
            ))
            info_layout.add_widget(opponent_layout)

            # Время
            time_layout = BoxLayout(size_hint_y=None, height=dp(40))
            time_layout.add_widget(Label(text="Time"))
            time_layout.add_widget(Label(
                text="03:45",
                font_size=dp(12),
                bold=True,
                color=(0.5, 0.5, 0.5, 1)
            ))
            info_layout.add_widget(time_layout)

            main_layout.add_widget(info_layout)

            # Разделитель
            divider = BoxLayout(size_hint_y=None, height=dp(1))
            with divider.canvas.before():
                Color((0.5, 0.5, 0.5, 1))
                Line(pos=[0, 0, 1, 0], width=1)
            main_layout.add_widget(divider)

            # Управление
            control_layout = BoxLayout(
                orientation='vertical',
                padding=[dp(20), dp(10)],
                size_hint_y=None,
                height=dp(100)
            )

            # Сдаться
            surrender_btn = Button(
                text="Surrender",
                background_color=(0.7, 0.2, 0.2, 1), # Красный
                color=(1, 1, 1, 1),
                font_size=dp(12),
                bold=True,
                size_hint_x=None,
                height=dp(40)
            )
            control_layout.add_widget(surrender_btn)
            surrender_btn.bind(on_press=self.surrender)

            # Анализ
            analyze_btn = Button(
                text="Analyze",
                background_color=(0.5, 0.2, 0.5, 1), # Фиолетовый
                color=(1, 1, 1, 1),
                font_size=dp(12),
                bold=True,
                size_hint_x=None,
                height=dp(40)
            )
            control_layout.add_widget(analyze_btn)
            analyze_btn.bind(on_press=self.analyze_moves)

            main_layout.add_widget(control_layout)

            # Доска
            board_container = BoxLayout(padding=[dp(20), 0])
            self.board = ChessBoard()
            board_container.add_widget(self.board)

            # Светлая темная доска
            light_color = (0.50, 0.50, 0.50, 1) # Темно-серый фон
            dark_color = (0.2, 0.2, 0.2, 1) # Темно-серый фон

            # Инициализация доски
            pieces = {}
            for row in range(8):
                for col in range(8):
                    color = light_color if (row + col) % 2 == 0 else dark_color
                    square = ChessSquare(
                        color,
                        size_hint=(None, None),
                        size=(dp(40), dp(40))
                    )
                    square.text = pieces.get((col, row))
                    if pieces.get((col, row)):
                        square.color = (0.5, 0.5, 0.5, 1) if color == light_color else (1, 1, 1, 1)
                    self.board.add_widget(square)

            board_container.add_widget(self.board)

            # Буквы
            letters_layout = BoxLayout(
                size_hint_y=None,
                height=dp(40),
                padding=[dp(20), 0, dp(20), 0]
            )
            for letter in ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']:
                letters_layout.add_widget(Label(
                    text=letter,
                    font_size=dp(12),
                    color=(0.5, 0.5, 0.5, 1),
                    size_hint=(None, None),
                    width=dp(40)
                ))
            main_layout.add_widget(letters_layout)

            # Таймер
            self.timer_running = True
            Clock.schedule_interval(self.update_timer, 1)

            self.add_widget(board_container)

            # Сдаться
            def surrender(self, instance):
                print("Surrender")
                self.timer_running = False
                self.canvas.clear()

            # Анализ
            def analyze_moves(self, instance):
                print("Analyze moves")
                self.canvas.clear()

            # Обновление таймера
            def update_timer(self, dt):
                if self.timer_running:
                    minutes = self.seconds // 60
                    seconds = self.seconds % 60
                    game_time = f"{minutes:02d}:{seconds:02d}"
                    self.game_time = game_time

            # Запуск таймера
            self.timer_running = False
            return super().on_start(*args)

    class GameApp(App):
        def build(self):
            sm = ScreenManager()
            sm.add_widget(GameScreen(name='game'))
            return sm

        def on_start(self):
            # Инициализация игры
            Window.size = (400, 600)
            if __name__ == '__main__':
                GameApp().run()

```

